Bounded Post-Deletion Inference with Weighted Dependencies (Extended Version)

Vishal Chakraborty
University of California, Irvine
vchakrab@uci.edu

Youri Kaminsky
Hasso Plattner Institute,
University of Potsdam
youri.kaminsky@hpi.de

Sharad Mehrotra
University of California, Irvine
sharad@ics.uci.edu

Felix Naumann
Hasso Plattner Institute,
University of Potsdam
felix.naumann@hpi.de

Sarvesh Pandey
Banaras Hindu University, Varanasi
University of California, Irvine
sarveshpandey@bhu.ac.in

Arnav Abhijit Dhariya
University of California, Irvine
dhariyaa@uci.edu

ABSTRACT

In databases, where data must adhere to semantics in the form of constraints (e.g., denial constraints, functional dependencies), simply deleting a record is often insufficient as it may be re-inferred from the remaining dependent data post-deletion. Recent work has explored strong deterministic notions of deletion that protect against such re-inferences. We categorize them as Semantic Erasure Against Leakage (Seal). Achieving Seal requires reasoning about dependencies to identify additional data that must be deleted to block re-inference after deletion, and preventing the deletion pattern from revealing the deleted value. Existing approaches enforce Seal deterministically, often leading to excessive deletion.

We propose Differential Seal (DS), a probabilistic relaxation of Seal. Rather than blocking all inference paths, DS ensures that no adversary, regardless of prior knowledge, gains more than bounded information about the deleted value from the deletion pattern and the remaining data. We capture dependencies as weighted rules, design a utility function that balances re-inference leakage against deletion cost, and develop mechanisms that satisfy DS. Experiments on real-world datasets show that DS substantially reduces extra deletion compared to deterministic baselines, guaranteeing Seal while maintaining quantifiable leakage bounds.

PVLDB Reference Format:
Vishal Chakraborty, Youri Kaminsky, Sharad Mehrotra, Felix Naumann, Sarvesh Pandey, and Arnav Abhijit Dhariya. Bounded Post-Deletion Inference with Weighted Dependencies
(Extended Version). PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Recent work [3, 10, 11, 13, 39], has explored strong notions of *data deletion* in databases with given semantic dependencies. They require that the remaining data must not enable re-inference of deleted values through such dependencies. These dependencies encode domain semantics, ETL and business rules, or are discovered from data itself [2, 9, 27, 36, 41].

ID	Zip	Symptom	Treatment	Diagnosis	Age	Gender	BMI	Test
t_1	94022	Fever	Tamiflu	?	45	M	28	72367-6
t_2	94022	Fever	Tamiflu	Flu	32	F	22	72367-6
t_3	92617	Fatigue	Metformin	Diabetes	45	M	28	4548-4
t_4	94301	Cough	Rest	Cold	55	F	30	72367-6
t_5	92617	Nausea	Metformin	Diabetes	45	M	28	4548-4

σ_1: Treatment predicts Diagnosis 95% of the time.
σ_2: Age, BMI predict Diagnosis for metabolic conditions 85% of the time.
σ_3: Symptom, Zip predict Diagnosis for infectious diseases 20% of the time.

Figure 1: Medical observations database D and dependencies.

Example 1.1. Database D in Fig. 1 records medical observations with attributes Zip, Symptom, Test, Diagnosis, Age, Gender, BMI, and Treatment, along with dependencies σ_1, σ_2 and σ_3. Suppose we wish to delete the cell $t_1[\text{Diagnosis}]$. Simply setting this cell to null does not prevent its value from being reconstructed: $t_1[\text{Treatment}] = \texttt{Tamiflu}$ enables inference via σ_1. In addition, tuple t_2, which shares Symptom and Zip with t_1, enables inference via σ_3.

Seal and its high deletion overheads. We refer to a deletion strategy that prevents the re-inference of deleted data from remaining data as satisfying *Semantic Erasure Against Leakage* (Seal). To delete a *target cell* c^*, a deletion mechanism outputs a set M of auxiliary cells, called *masks*, to delete in addition to c^*. Seal requires a deterministic mask M_{det} that blocks *all* inference paths to c^*, so that the remaining data $D \setminus (M_{\text{det}} \cup \{c^*\})$ reveals nothing about the value of c^*. Prior work on Seal includes deleting all dependent data [3, 13] or identifying minimal such masks via ILP formulations and heuristics [11, 30] to render dependencies unusable.

A key weakness of existing approaches is that they assume all dependencies to always hold. In practice, however, many constraints, especially those that are discovered automatically, hold only approximately [9, 26, 29, 33, 35, 41]. Such approximate dependencies carry weights reflecting how often they facilitate true inference. In Example 1, dependency σ_1 enables correct inference 95% of the time, σ_2 85%, and σ_3 20%. Existing approaches treat all dependencies as deterministic (i.e., they enable correct inference 100% of the

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

time), blocking all inference paths regardless of weight, leading to an undesirable number of auxiliary deletions.

Another issue arises when constraints are *value-dependent [6],* such as conditional functional dependencies [15] or semantic integrity constraints [27]. In our example, inference through σ_2 is triggered only when the condition is *metabolic*, while inference through σ_3 applies only to *infectious* diseases. In such settings, preventing re-inference requires masking cells in a value-dependent manner. For instance, if t_1[Diagnosis] = `Flu`, blocking inference requires masking the cells corresponding to Symptom/Zip. If instead t_1[Diagnosis] = `Diabetes`, then the cells corresponding to Age/BMI must be masked. However, this creates a new leakage channel: the *choice of which cells are masked* reveals information about the deleted value itself. An adversary observing that Symptom/Zip are masked can infer that the diagnosis was infectious; masking Age/BMI suggests a metabolic condition. Thus, leakage arises not only from what remains unmasked (i.e., residual inference), but also from the masking pattern itself. Deterministic approaches avoid this by instantiating all conditional constraints irrespective of the actual value, thereby masking the union of all potentially implicated cells. In our example, this would require masking both Symptom/Zip and Age/BMI, making M_{det} strictly larger — often unnecessarily so.

τ-Seal: a relaxation of Seal. We propose *τ-Differential Seal* (τ-Seal), a principled relaxation that exploits the probabilistic nature of approximate dependencies to reduce auxiliary deletions while bounding what any adversary can learn. Under Seal, a deletion mechanism producing a mask M_{det} blocks all inference paths, and the best an adversary can do is guess. Their success probability is some baseline determined by the data distribution. Contrastingly, under τ-Seal, a mechanism produces a comparatively smaller mask M_τ and guarantees that the adversary's probability of correctly inferring the deleted value is at most τ. The larger the tolerable leakage, i.e., the parameter τ, the smaller the mask M_τ may be, accepting a controlled increase in inference risk in exchange for preserving more data. The parameter τ governs this tradeoff directly as a probability, providing an interpretable privacy guarantee.

Ensuring τ-Seal requires mechanisms to bound leakage in the post-deletion database $D \setminus (M_\tau \cup \{c^*\})$. Such mechanisms must address both: reinference from residual data via dependencies and the choice of which cells constitute the mask (i.e., mask patterns).

Re-inference leakage. Inference can chain transitively: an adversary might first infer an intermediate masked cell via one dependency, then use that cell to activate another dependency that reaches the deleted cell c^*. Multiple chains may exist under a given mask, but they can share intermediate cells. If the adversary fails to infer a shared cell, all chains depending on it fail together. Naive aggregation using the union bound overestimates leakage by assuming independence of all chains. We formulate *re-inferential leakage* to capture the adversary's total re-inference probability and introduce a *mask-disjoint decomposition* that avoids overcounting by selecting chains with non-overlapping masked intermediates (Sec. 3).

Masking privacy. Bounding the mask pattern channel requires comparing the mechanism's output across databases that differ in the value of the deleted cell c^*. We formulate *masking privacy*, inspired by differential privacy (DP) [14], to bound this channel. Standard DP defines *neighbors* as databases differing in a single tuple, assuming tuple independence, an assumption that breaks under correlations [21]. While edge and node DP [20] and the Pufferfish framework [22] generalize this, they do not address settings, like ours, where changing one cell necessitates cascading changes to maintain consistency with constraints. Consider again σ_3, which links the diagnoses of t_1 and t_2; changing t_1[Diagnosis] from `Flu` to `Cold` (see Fig. 2) forces t_2[Diagnosis] to change as well. The resulting databases differ in multiple cells across multiple tuples. We define neighboring databases with respect to the target cell to capture these cascades, and require the mask distribution to be similar across all such pairs.

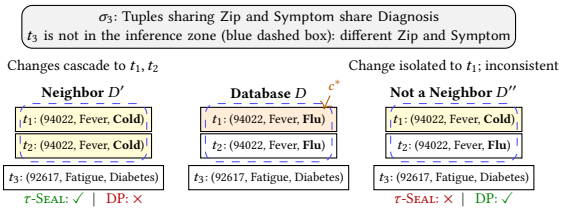


Figure 2: Neighboring databases under τ-Seal.

Composing the leakage channels. The two channels contribute independently to the adversary's advantage. We show that re-inferential leakage and masking privacy compose on the log-odds scale, allowing one to set a single privacy threshold τ.

τ-Seal mechanisms. To achieve τ-Seal, we use the exponential mechanism [31] (traditionally used for DP), sampling masks proportional to a utility function that balances re-inferential leakage against deletion cost. We additionally provide a greedy Gumbel-max based mechanism [21] that constructs masks incrementally without exhaustive enumeration. We evaluate on four real-world and one artificial dataset, measuring leakage, deletion volume, and runtime against a deterministic baseline.

Contributions. In summary, in this paper, we:

(1) formalize τ-Seal, a privacy guarantee for deletion under weighted dependencies that bounds the adversary's probability of correctly inferring the deleted value, parameterized by a single interpretable threshold τ;
(2) formulate *re-inferential leakage* to measure the adversary's re-inference probability, and introduce mask-disjoint decomposition to compute it without overcounting;
(3) introduce *masking privacy* for constraint-rich settings where changing one cell may require cascading changes across tuples, define appropriate neighboring databases;
(4) design two mechanisms, an exponential mechanism and a progressive variant, that balance leakage against deletion cost;
(5) evaluate on five datasets, showing deletion reduction over the deterministic baseline while maintaining bounded privacy loss.

Sec. 2 formalizes the data model and τ-Seal guarantee. Sec. 3 develops re-inference leakage analysis. Sec. 4 addresses mask pattern leakage, neighboring databases, and channel composition. Sec. 5 presents our mechanisms which we evaluate in Sec. 6. We conclude with related work in Sec. 7 and future work in Sec. 8.

2 FORMALIZING DIFFERENTIAL SEAL

We introduce the data model, dependencies, and deletion masks (Sec. 2.1) and then state the τ-Seal guarantee and problem (Sec. 2.2).

2.1 Data, Dependencies, and Masks

Data. We consider a relational database instance D consisting of tuples $\{t_1, \ldots, t_n\}$ over a schema $\mathcal{S} = \{A_1, \ldots, A_m\}$. For ease of presentation, we assume that D has only one relation and every tuple has an ID. For tuple t_i and attribute A_j, we write $t_i[A_j] \in \mathsf{dom}(A_j)$ for the cell's value and $c_{i,j}$ for the cell itself. A deletion request is for a specific *target cell* $c^* = t_\ell[A_k]$. In Example 1.1 , the schema $\mathcal{S} = \{\mathsf{ID}, \mathsf{Zip}, \mathsf{Symptom}, \mathsf{Test}, \mathsf{Diagnosis}, \mathsf{Age}, \mathsf{Gender}, \mathsf{BMI}, \mathsf{Treatment}\}$. The target cell $c^* = t_1[\mathsf{Diagnosis}]$.

Dependencies. We express data dependencies by extending *relational dependency rules* (RDRs) [11]. As claimed in [11], these rules express a large number of dependencies and constraints, including functional dependencies (FDs), conditional FDs, denial constraints (DCs), conditional DCs, etc. To express approximate dependencies, we associate a *weight* with each RDR.

Definition 2.1 (Weighted RDR). *A* weighted RDR, *denoted* $\sigma = (\Delta, Q, w_\sigma)$, *over schema* $\mathcal{S}$ *comprises a dependency* $\Delta : A(\mathbf{X}) \not\perp A_1(\mathbf{X}_1), \ldots, A_p(\mathbf{X}_p)$ *with* head $A(\mathbf{X})$ *and* tail $\{A_1(\mathbf{X}_1), \ldots, A_p(\mathbf{X}_p)\}$, *a condition query* Q, *and weight* $w_\sigma \in (0, 1]$. *For tuples satisfying query* Q, *an adversary knowing the remaining cell values correctly infers the remaining cell's value with probability* w_σ.

Fig. 3 shows the weighted RDRs $\sigma_1, \sigma_2, \sigma_3$ in our running example.

RDR (σ)	Specification	w_σ
σ_1	Dep (Δ): Diagnosis($\mathbf{X}$) $\not\perp$ Treatment($\mathbf{X}$) Cond (Q): `SELECT rid AS X FROM D`	0.95
σ_2	Dep (Δ): Diagnosis($\mathbf{X}$) $\not\perp$ Age($\mathbf{X}$), BMI($\mathbf{X}$) Cond (Q): `SELECT rid AS X FROM D` `WHERE Diagnosis IN (Metabolic Conditions)`	0.85

σ_3	Dep (Δ): Diagnosis($\mathbf{X}_1$) $\not\perp$ Diagnosis($\mathbf{X}_2$)
	Cond (Q): `SELECT` t_1`.rid AS` $\mathbf{X}_1$, t_2`.rid AS` $\mathbf{X}_2$ `FROM D` t_1, `D` t_2
	`WHERE` t_1`.Symptom =` t_2`.Symptom AND` t_1`.Zip =` t_2`.Zip`
	`AND` t_1`.Diagnosis IN (Infectious Diseases)`
	`AND` t_1`.rid ≠` t_2`.rid`
		0.20

Figure 3: Weighted RDRs for the running example.

To reason about dependencies at the level of individual cells in a database D, we *instantiate* the weighted RDRs by evaluating condition Q on D and substituting the returned tuple identifiers.

Definition 2.2 (Instantiated Weighted RDR). *An* instantiation *of a weighted RDR* σ *is obtained by evaluating* Q *on* D, *yielding tuple bindings* $(t_i, t_{i_1}, \ldots, t_{i_p})$, *and substituting these for* $(\mathbf{X}, \mathbf{X}_1, \ldots, \mathbf{X}_p)$: $\hat{\sigma} : t_i[A] \not\perp t_{i_1}[A_1], \ldots, t_{i_p}[A_p]$ *with weight* w_σ.

We write $\mathsf{Head}(\hat{\sigma}) = \{t_i[A]\}$, $\mathsf{Tail}(\hat{\sigma}) = \{t_{i_1}[A_1], \ldots, t_{i_p}[A_p]\}$, and $\mathsf{Cells}(\hat{\sigma}) = \mathsf{Head}(\hat{\sigma}) \cup \mathsf{Tail}(\hat{\sigma})$. Given a set Σ of RDRs and database D, we write $\hat{\Sigma}(D)$ for the set of all instantiated RDRs. Weighted RDRs subsume approximate FDs [35, 41] (weight $w < 1$, universal condition), conditional FDs [15] (condition Q encodes the predicate), similarity dependencies [9, 26], and exact constraints such as keys ($w = 1$); see [1, 11] for details on extraction from data.

Example 2.1. In Fig. 3, for σ_1, each tuple in Fig. 1 yields an instantiation: $\hat{\sigma}_1^{(1)} : t_1[\mathsf{Diagnosis}] \not\perp t_1[\mathsf{Test}]$ with $w = 0.95$, and similarly for $t_2, \ldots, t_5$. For σ_3, tuples t_1 and t_2 share both Symptom and Zip. If $t_1[\mathsf{Diagnosis}]$ is an infectious disease, the condition is satisfied, yielding: $\hat{\sigma}_3^{(1,2)} : t_1[\mathsf{Diagnosis}] \not\perp t_2[\mathsf{Diagnosis}]$ ($w = 0.20$). If instead $t_1[\mathsf{Diagnosis}]$ is a metabolic condition, σ_3's condition is not satisfied, and these instantiations do not exist.

Deletion masks. To delete a target cell c^*, a deletion mechanism may need to delete cells in addition to c^* whose values would otherwise enable re-inference of c^*.

Definition 2.3 (Deletion Mask). *Given a target cell* c^* *in database* D, *a* deletion mask *is a set* $M \subseteq D \setminus \{c^*\}$ *of auxiliary cells to be deleted (set to Null) in addition to* c^*.

A deletion mechanism $\mathbb{M} : (D, c^*) \to M$ takes as input the target cell c^* and the database D and produces a mask M and the post-deletion database $D \setminus (M \cup \{c^*\})$ sets c^* and every cell in M to `Null`, leaving all other cells unchanged. We write $D \setminus M$ for $D \setminus (M \cup \{c^*\})$ when c^* is clear from context.

Definition 2.4. *Given a target cell* c^* *in a database* D, *a mechanism satisfying* Semantic Erasure Against Leakage *(Seal) produces a deterministic mask* M_{det} *that deletes all dependent data of* c^*.

Thus, the mask M_{det} contains all cells such that $D \setminus M_{\text{det}}$ reveals nothing about c^*'s value. We relax this requirement next.

2.2 The τ-Seal Guarantee

For a target cell c^*, we want to bound the adversary's probability of re-inferring the deleted value of c^* successfully while potentially having a deletion mechanism produce a smaller mask. Let P denote the *baseline inference probability* (as achieved by Seal):

$$P = \Pr[D[c^*] = v | D \setminus M_{\text{det}}], \tag{1}$$

the probability that the adversary correctly guesses the value of c^* given the post-deletion database under deterministic Seal. Since M_{det} masks all dependent data, P reflects only the adversary's background knowledge (e.g., value frequencies). Under any mechanism that produces a smaller mask, some dependent data remains visible, and the adversary may exceed P; our goal is to bound this excess.

Definition 2.5. *Given a target cell* c^* *in database* D *with value* $v = D[c^*]$, *and* $\tau \in (0, 1)$, *a deletion mechanism* $\mathbb{M}$ *satisfies* τ-Seal *if for the mask* M *produced by* $\mathbb{M}$, *we have* $\Pr[D[c^*] = v \mid D \setminus M] \leq \tau$.

Smaller τ, (while $\tau > P$; for $\tau \leq P$, we recover Seal) provides stronger privacy. As τ increases, the mechanism may produce smaller masks, accepting a controlled increase in inference risk in exchange for preserving more data.

2.3 Problem Formalization & Solution Overview

In this work, we consider randomized deletion mechanisms that satisfy τ-Seal. We formalize our problem as follows.

Problem. Given a database D, a set Σ of weighted RDRs, a target cell c^*, and a privacy threshold $\tau \in (0, 1)$, design a randomized mechanism $\mathbb{M} : (D, c^*) \to M$ that satisfies τ-Seal (Defn. 2.5) while minimizing the expected mask size $\mathbb{E}[|M|]$.

Solution overview. An adversary observing $D \setminus M$ gains information from two sources: the *mask pattern*, i.e., which cells are Null, may reveal information about c^* (Sec. 4), and the *remaining*

dependent data may enable re-inference through the dependencies (Sec. 3). We analyze these channels separately and show how they comprise the total guarantee τ.

Composability. We formalize the bound via the *odds* of correctly guessing the value of c^*. For any event E with $\Pr[E] = p$, the odds of E are $\text{odds}(E) = p/(1-p)$. The τ-Seal guarantee implies the posterior odds not to exceed $\tau/(1-\tau)$, while the baseline odds are $P/(1-P)$. By Bayes' theorem, the posterior odds given the adversary's observation $D \setminus (M \cup \{c^*\})$ are:

$$\frac{\Pr[c^* = v \mid D \setminus M]}{\Pr[c^* \neq v \mid D \setminus M]} = \frac{P}{1-P} \times \frac{\Pr[D \setminus M \mid c^* = v]}{\Pr[D \setminus M \mid c^* \neq v]} \tag{2}$$

The likelihood ratio on the right measures how much the observation $D \setminus (M \cup \{c^*\})$ shifts the odds beyond the baseline. We decompose it via the chain rule. The mechanism first selects which cells to mask (producing mask M), then the visible values $D_{\text{vis}} = D \setminus (M \cup \{c^*\})$ are determined by M and the underlying data:

$$\frac{\Pr[D \setminus M \mid c^* = v]}{\Pr[D \setminus M \mid c^* \neq v]} = \underbrace{\frac{\Pr[M \mid c^* = v]}{\Pr[M \mid c^* \neq v]}}_{\text{mask channel}} \times \underbrace{\frac{\Pr[D_{\text{vis}} \mid M, c^* = v]}{\Pr[D_{\text{vis}} \mid M, c^* \neq v]}}_{\text{re-inference channel}} \tag{3}$$

The first factor captures how much the *selection of which cells to delete* depends on the value of c^*: if the mechanism chooses different masks for different values, this ratio is not 1. The second factor captures how much the *visible dependent data* enables re-inference of the value of c^* through the dependencies: if the mask leaves dependent data unmasked, this ratio is large.

Taking logarithms converts the multiplicative decomposition into an additive *privacy budget*. Define the log-odds gain of each channel as: $\varepsilon_m = \ln \frac{\Pr[M|c^*=v]}{\Pr[M|c^*\neq v]}$ and $\varepsilon_r = \ln \frac{\Pr[D_{\text{vis}}|M,c^*=v]}{\Pr[D_{\text{vis}}|M,c^*\neq v]}$. Combining Eqns. (2) and (3) and taking logarithms:

$$\underbrace{\ln \frac{\tau}{1-\tau}}_{\text{max. allowed log-odds}} \geq \underbrace{\ln \frac{P}{1-P}}_{\text{baseline log-odds}} + \varepsilon_m + \varepsilon_r \tag{4}$$

The re-inference channel shifts the adversary from the baseline probability P to the re-inference probability $\mathcal{L}(M)$, so its log-odds contribution is $\varepsilon_r = \ln \frac{\mathcal{L}(M)(1-P)}{(1-\mathcal{L}(M))P} = \text{logit}(\mathcal{L}(M)) - \text{logit}(P)$. Substituting into Eqn. (4), the baseline log-odds cancel:

$$\text{logit}(\tau) \geq \cancel{\text{logit}(P)} + \varepsilon_m + \text{logit}(\mathcal{L}) - \cancel{\text{logit}(P)} = \varepsilon_m + \text{logit}(\mathcal{L}).$$

Thus the τ-Seal guarantee depends only on the mask-privacy budget ε_m and the re-inference probability $\mathcal{L}(M)$. Inverting, we obtain the maximum tolerable re-inference probability:

$$L_0 = \frac{\tau\, e^{-\varepsilon_m}}{1 - \tau + \tau\, e^{-\varepsilon_m}}. \tag{5}$$

The user specifies a single privacy threshold $\tau \in (P, 1)$; setting $\tau \leq P$ recovers deterministic Seal. Given a system-level mask-privacy parameter ε_m, the re-inference threshold L_0 follows from Eqn. (5). When $L_0 = 0$, no re-inference is tolerated, and the mechanism must produce the deterministic mask M_{det}, recovering Seal.

As τ increases, L_0 grows, permitting smaller masks at controlled re-inference risk. The mechanism must ensure that $\mathcal{L}(M) \leq L_0$, where $\mathcal{L}(M)$ is the probability of correctly re-inferring the value of c^* from visible dependent data through the dependencies (Sec. 3). We formalize each channel and show that $\mathcal{L}(M) \leq L_0$ together with ε_m-masking privacy implies the τ-Seal guarantee (Sec. 4).

3 RE-INFERENCE LEAKAGE

We present dependency hypergraphs (see Sec. 3.1). Inference in our framework propagates through masked cells in dependency hypergraphs via inference chains introduced (see Sec. 3.2). We discuss the aggregation of independent inferences (see Sec. 3.3). Finally, we define re-inference leakage (see Sec. 3.4).

3.1 Dependency Hypergraph

Instantiated RDRs induce inference relationships among cells: if an adversary knows all but one cell participating in an instantiated RDR $\hat{\sigma}$, the remaining cell can be inferred with probability w_σ. We capture these relationships using a hypergraph.

An instantiated RDR $\hat{\sigma} \in \hat{\Sigma}(D)$ is *connected to a cell* c^* if there exists a sequence $\hat{\sigma}_1, \ldots, \hat{\sigma}_k = \hat{\sigma}$ of instantiated RDRs such that $c^* \in \text{Cells}(\hat{\sigma}_1)$ and $\text{Cells}(\hat{\sigma}_i) \cap \text{Cells}(\hat{\sigma}_{i+1}) \neq \emptyset$ for all $i < k$.

Definition 3.1. *Given a set Σ of weighted RDRs over a database D and a target cell c^*, the* dependency hypergraph*, denoted $\mathcal{H}_D^{c^*} = (V, E)$, consists of the set E of hyperedges containing one hyperedge $e = \text{Cells}(\hat{\sigma})$ with weight w_σ for each instantiated RDR* connected *to c^*, and the set of vertices $V = \bigcup_{e \in E} e$.*

The hypergraph is undirected: each hyperedge represents an inference relationship exploitable in any direction. An inference step via edge e applies when the adversary knows all but one cell in e; the remaining cell can then be inferred with probability w_e. Inference can be direct (a single edge connects known cells to the target) or transitive (a sequence of edges towards the target).

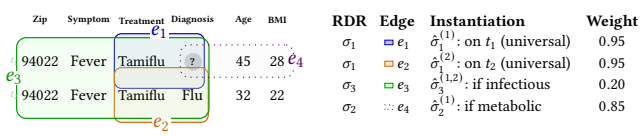


Figure 4: Dependency hypergraph (partially depicted).

Example 3.1. Fig. 4 shows a partial hypergraph on tuples t_1 and t_2 and the corresponding instantiated weighted RDRs. The hypergraph consists of hyperedge e_4 if $t_1[\text{Diagnosis}]$ is a metabolic condition and hyperedge e_3 only if $t_1[\text{Diagnosis}]$ is an infectious disease.

3.2 Inference Chains Given Masks

Given a mask, the values of the cells in the mask are replaced with Null. We define *inference chains* to capture the operational sequence of inferences given a mask, where the adversary must recover masked intermediate cells before later edges become active. We begin by formalizing when a hyperedge can be used.

Definition 3.2 (Active Hyperedge). *Given a database D, target cell c^*, and the induced hypergraph $\mathcal{H}_D^{c^*} = (V, E)$, for a hyperedge $e \in E$, cell $x \in e$, and mask M, we say e is* active *for inferring x under M if all cells in $e \setminus \{x\}$ are visible: $e \setminus \{x\} \subseteq V \setminus (M \cup \{c^*\})$.*

For $c^* = t_1[\text{Diagnosis}]$, in Fig. 5 with $M_1 = \emptyset$, the hyperedge $e_1 = \{\text{Treatment}, \text{Diagnosis}\}$ is active for inferring Diagnosis since Treatment is visible. Similarly, $e_2 = \{\text{Age}, \text{BMI}, \text{Diagnosis}\}$ is active

since both Age and BMI are visible. For $M_2 = \{\text{Treatment}\}$, edge e_1 is no longer active for Diagnosis since Treatment is masked.

To formalize *transitive* inference, we close the initially visible cells under repeated applications of active hyperedges, yielding all cells that become visible to the adversary by iterated inference.

Given a dependency hypergraph $\mathcal{H} = (V, E)$, let $K_0(M) := V \setminus (M \cup \{c^*\})$ be the set of initially visible cells under mask M. We define derived knowledge as $K_{t+1}(M) := K_t(M) \cup \{x \in V : \exists e \in E \text{ s.t. } x \in e \text{ and } e \setminus \{x\} \subseteq K_t(M)\}$. Let $K^*(M)$ denote the least fixpoint of this process. We say a cell is *(transitively) inferable* under M if it is in $K^*(M)$.

For $M_4 = \{\text{Treatment}, \text{Age}\}$ in Fig. 5, we have $K_0 = \{\text{Gender}, \text{BMI}\}$. Since $e_a = \{\text{Gender}, \text{Age}\}$ has Gender inferable, the adversary can infer Age, giving $K_1 = \{\text{Gender}, \text{BMI}, \text{Age}\}$. Now $e_c = \{\text{Age}, \text{Treatment}\}$ becomes active, allowing inference of Treatment. Therefore, $K_2 = \{\text{Gen}, \text{BMI}, \text{Age}, \text{Treatment}\}$. The closure continues until reaching Diagnosis. Each step of the closure corresponds to one hyperedge in a *sequence of hyperedges* which the adversary follows to reach c^* through masked cells.

Definition 3.3. *An* inference chain *to target c^* under mask M is a sequence of distinct hyperedges $p = e_1\text{-}e_2\text{-}\cdots\text{-}e_k$ such that:*

(1) $c^* \in e_k$;
(2) for each $i < k$, $e_i \cap e_{i+1} \neq \emptyset$ with a shared witness cell $x_i \in e_i \cap e_{i+1}$;
(3) e_1 is active for inferring x_1 under M (Definition 3.2); and
(4) for all $j > 1$, e_j is not *active for inferring x_j with respect to the initial visibility $K_0(M)$ (where $x_k = c^*$), but becomes active after earlier inferences along the chain, i.e., there exist knowledge sets $K_0(M) \subseteq K_1 \subseteq \cdots \subseteq K_{k-1} \subseteq K^*(M)$ with $K_i = K_{i-1} \cup \{x_i\}$ and $e_i \setminus \{x_i\} \subseteq K_{i-1}$ for all $i \in [k]$.*

The chain weight *is $w(p) = \prod_{i=1}^{k} w_{e_i}$. The* masked intermediate cells *are Masked$(p) = \{x_i : i < k, x_i \in M\}$.*

Condition (4) ensures maximality. If some edge e_j were already active at the set $K_0(M)$, the adversary would use the shorter chain $e_j\text{-}\cdots\text{-}e_k$ instead.

Example 3.2. Consider mask $M_4 = \{\text{Treatment}, \text{Age}\}$ in Fig. 5. Neither e_1 nor e_2 is initially active (both require masked cells). Two valid chains reach Diagnosis: $p_1 = e_a\text{-}e_c\text{-}e_1$ infers Age via e_a, then Treatment via e_c, then Diagnosis via e_1, with weight $w(p_1) = 0.90 \times 0.88 \times 0.95 = 0.752$ and $\text{Masked}(p_1) = \{\text{Age}, \text{Treatment}\}$. Chain $p_2 = e_a\text{-}e_2$ infers Age via e_a, then Diagnosis via e_2 (since BMI is already visible), with weight $w(p_2) = 0.90 \times 0.85 = 0.765$ and $\text{Masked}(p_2) = \{\text{Age}\}$.

3.3 Aggregating Leakage

Given the set of valid inference chains $\mathcal{P}(M)$ under mask M, we compute the probability that an adversary successfully re-infers the deleted value $D[c^*]$. We model each hyperedge e as an independent Bernoulli trial that succeeds with probability w_e; a chain p succeeds if all its hyperedges succeed, so $\Pr[p \text{ succeeds}] = w(p)$.

An adversary observes the released database (with masked cells set to NULL) and reasons through dependency paths to reconstruct $D[c^*]$. Each chain $p \in \mathcal{P}(M)$ represents a line of reasoning: the adversary infers masked intermediate cells step by step, then applies a final rule to predict c^*. When multiple chains are available, the adversary can cross-validate: if two independent lines of reasoning (i.e., inference chains) point to the same value, the adversary's confidence increases beyond either chain alone.

However, chains sharing a masked intermediate cell m are *not* independent. They both depend on correctly inferring m's value, so failure to infer m defeats both simultaneously. Among chains sharing the masked cell m, only the strongest provides additional evidence; the rest are redundant. We formalize this via *mask-disjointness*. Formally. Two chains $p, p' \in \mathcal{P}(M)$ are *mask-disjoint* if $\text{Masked}(p) \cap \text{Masked}(p') = \emptyset$. Mask-disjoint chains rely on inferring disjoint sets of masked cells, so their successes are independent.

In Example 3.2, chains p_1 and p_2 both require inferring Age, so $\text{Masked}(p_1) \cap \text{Masked}(p_2) = \{\text{Age}\} \neq \emptyset$. Thus, they are not mask-disjoint. In contrast, under $M_2 = \{\text{Treatment}\}$, chains e_2 (Masked $= \emptyset$) and $e_c\text{-}e_1$ (Masked $= \{\text{Treatment}\}$) are mask-disjoint: they provide genuinely independent evidence about c^*.

3.4 Re-inference Leakage

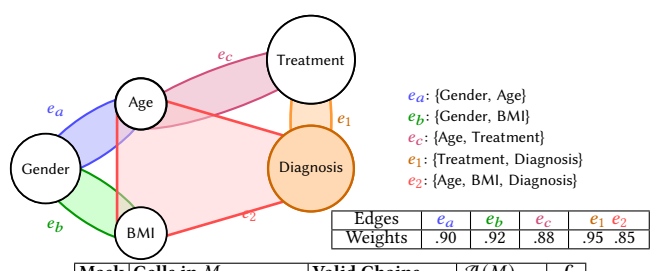
We aggregate leakage by identifying the adversary's strongest independent lines of reasoning, i.e., inference chains.

Given database D, target c^*, and mask M, let $\mathcal{P}(M)$ denote the valid inference chains to c^* under M, ordered by decreasing weight: $w(p_1) \geq w(p_2) \geq \cdots$. The *independent evidence set* $\mathcal{A}(M)$ consists of chains providing non-redundant evidence, constructed by including p_i only if it shares no masked intermediates with any stronger included chain: $\mathcal{A}(M) = \left\{p_i : \text{Masked}(p_i) \cap \bigcup_{p_j \in \mathcal{A}, \mathbb{M} j<i} \text{Masked}(p_j) = \emptyset\right\}$. The *re-inference leakage* is the probability that at least one independent line of reasoning succeeds:

$$\mathcal{L}(M, c^*, D) = 1 - \prod_{p \in \mathcal{A}(M)} \big(1 - w(p)\big). \tag{6}$$

We write $\mathcal{L}(M)$ when c^* and D are clear from context. When $M = \mathcal{I}(c^*)$ (all inference zone cells masked), no chains survive and $\mathcal{L} = 0$. When $M = \emptyset$, all chains are active and $\mathcal{L}$ is maximized.

Each chain in $\mathcal{A}$ provides an independent piece of evidence for c^*'s value: its success depends on masked cells that no other chain in $\mathcal{A}$ shares. When chains compete for the same masked cell, only the strongest survives—weaker chains through the same cell



Edges	e_a	e_b	e_c	e_1	e_2
Weights	.90	.92	.88	.95	.85

Mask	Cells in M	Valid Chains	$\mathcal{A}(M)$	$\mathcal{L}$
M_1	$\emptyset$	e_1, e_2	e_1, e_2	.992
M_2	{Treatment}	$e_c{-}e_1, e_2$	$e_2, e_c{-}e_1$	.975
M_3	{Age}	$e_1, e_a{-}e_2$	$e_1, e_a{-}e_2$	.988
M_4	{Treatment, Age}	$e_a{-}e_c{-}e_1, e_a{-}e_2$	$e_a{-}e_2$	.765
M_5	{Gen}	e_1, e_2	e_1, e_2	.992
M_6	{Age, BMI}	e_1	e_1	.950
M_7	{Treatment, Age, BMI}	$e_a{-}e_c{-}e_1$	$e_a{-}e_c{-}e_1$	.752
M_8	all four cells	none	$\emptyset$	0

Figure 5: Masks on hypergraph $\mathcal{H}_{ex}$ for $c^* = t_1[\textbf{Diagnosis}]$.

add no evidence beyond what the strongest already provides. The noisy-OR aggregation (Eqn. 6) is the standard combination rule for independent Bernoulli evidence sources: the adversary succeeds unless *every* independent line of reasoning fails.

Example 3.3. Under $M_2 = \{\text{Treatment}\}$ (Fig. 5): chain e_2 ($w = 0.85$, Masked $= \emptyset$) and e_c-e_1 ($w = 0.836$, Masked $= \{\text{Treatment}\}$) are mask-disjoint—two independent lines of reasoning through different rules. Both are included in $\mathcal{A}$: $\mathcal{L}(M_2) = 1 - (1 - 0.85)(1 - 0.836) = 0.975$. The adversary benefits from cross-validation: even if the stronger path (e_2) fails, the weaker path (e_c-e_1) may independently succeed. Under $M_4 = \{\text{Treatment}, \text{Age}\}$: three chains exist—$p_2 = e_a$-$e_2$ ($w = 0.765$, Masked $= \{\text{Age}\}$), $p_1 = e_a$-e_c-e_1 ($w = 0.752$, Masked $= \{\text{Age}, \text{Treatment}\}$), and $p_3 = e_a$-e_c-e_2 ($w = 0.673$, Masked $= \{\text{Age}, \text{Treatment}\}$). All three require inferring Age: the adversary's ability to recover Age determines whether *any* path succeeds. The strongest chain p_2 captures the best evidence obtainable through Age; p_1 and p_3 depend on the same uncertain inference and add no independent evidence. Thus $\mathcal{A} = \{p_2\}$ and $\mathcal{L}(M_4) = 0.765$, reflecting that all attack paths funnel through the same bottleneck.

4 MASK PATTERN LEAKAGE

The second leakage channel arises from the mask pattern itself: the choice of which cells to delete may reveal c^*'s value. We construct value-independent structures needed to compare masks across databases (Sec. 4.1), define neighboring databases (Sec. 4.2), and formulate a utility function with bounded sensitivity for the exponential mechanism (Sec. 4.3). Finally, we prove that the two leakage channels compose into the τ-Seal guarantee (Sec. 4.4).

4.1 Maximum Hypergraph

The dependency hypergraph $\mathcal{H}_D^{c^*}$ varies with the value of c^*, since different values activate different conditional RDRs (Example 3.1). To define a mask space and a neighborhood relation that are independent of c^*'s value, we construct a hypergraph that encompasses all edges under any value assignment.

For hypergraphs $\mathcal{H}_1 = (V_1, E_1)$ and $\mathcal{H}_2 = (V_2, E_2)$, we write $\mathcal{H}_1 \subseteq \mathcal{H}_2$ if $V_1 \subseteq V_2$ and $E_1 \subseteq E_2$, and define $\mathcal{H}_1 \cup \mathcal{H}_2 = (V_1 \cup V_2, E_1 \cup E_2)$.

Definition 4.1 (Maximum Hypergraph). *Given weighted RDRs Σ, database D, and target cell c^*, the* maximum hypergraph *is $\mathcal{H}_{\max}^{c^*} = \bigcup_{v \in \mathtt{dom}(c^*)} \mathcal{H}_{D[c^* \leftarrow v]}^{c^*}$ where $D[c^* \leftarrow v]$ denotes D with c^* set to v (and any cascading changes for consistency).*

By construction, $\mathcal{H}_D^{c^*} \subseteq \mathcal{H}_{\max}^{c^*}$ for every value of c^*, and $\mathcal{H}_{\max}^{c^*}$ is identical across all neighbors. We refer to the set of vertices in $\mathcal{H}_{\max}^{c^*}$ as the *inference zone* and denote it $\mathcal{I}(c^*)$.

This invariance is essential for privacy. Since $\mathcal{H}_D^{c^*}$ varies across neighbors, a deterministic mechanism that selects the optimal mask for each hypergraph would leak the value of c^* through the deletion pattern itself. The mechanism must therefore draw masks from a space that is fixed across all neighbors. The candidate mask space $2^{\mathcal{I}(c^*) \setminus \{c^*\}}$ satisfies this requirement: it is determined by $\mathcal{H}_{\max}^{c^*}$, which is identical regardless of c^*'s value. Our deletion mechanisms draw from this fixed space but evaluates each candidate's quality, i.e., how well it prevents re-inference and how many auxiliary cells it deletes, on the actual hypergraph $\mathcal{H}_D^{c^*}$.

Example 4.1. Returning to Fig. 4, the maximum hypergraph $\mathcal{H}_{\max}^{c^*}$ includes both e_3 (infectious values) and e_4 (metabolic values), since each arises under some value of c^*. The inference zone $\mathcal{I}(c^*)$, therefore, contains all cells reachable through either edge, yielding a fixed set regardless of the actual value being deleted.

4.2 Neighboring Databases

Bounding mask pattern leakage requires comparing the mechanism's output across databases that differ in the value of the target cell c^*.' The set of neighbors determines the strength of the guarantee: a larger set yields a stronger bound that remains valid if the adversary narrows the plausible alternatives to a subset.

In standard differential privacy, neighbors differ in exactly one tuple. Our setting differs in two ways. First, changing c^* may propagate through weighted dependencies in Σ, causing multiple cells across multiple tuples within $\mathcal{I}(c^*)$ to change; the set of neighbors is therefore *wider*. Second, only changes confined to $\mathcal{I}(c^*)$ qualify; cells outside the inference zone are structurally independent of c^* and must remain fixed, i.e., the neighbor set is therefore *structured*.

We impose no restriction on how cells within $\mathcal{I}(c^*)$ may change, and no constraint satisfaction condition on neighbors. Since RDRs are weighted approximate dependencies, D may already violate some constraints, and the adversary cannot use binary satisfaction checks to eliminate candidate values of c^*: violations are expected, not informative. The adversary's ability to exploit dependency patterns is modulated by RDR weights and captured entirely by the re-inference leakage $\mathcal{L}(M, c^*, D)$ (Defn. **??**), which is evaluated over the full post-deletion dataset. In particular, the neighbor set includes the minimal-change case where only c^* differs and all other cells in $\mathcal{I}(c^*)$ are identical. Note that this scenario most advantageous to the adversary, since it maximizes the shared context available for distinguishing values.

Definition 4.2. *Given database D over schema $\mathcal{S}$ and target cell c^*, we say database D' is a* neighbor *of D, and write $D \sim_{c^*} D'$, if*

(1) $D[c^*] \neq D'[c^*]$, and
(2) $D[c] = D'[c]$ for all cells $c \notin \mathcal{I}(c^*)$.

Example 4.2. If in D (Fig. 1) $t_1[\text{Diagnosis}] = \mathtt{Flu}$, then D' (Fig. 2) with $t_1[\text{Diagnosis}] = \mathtt{Cold}$ is a neighbor. Since σ_3 links the diagnoses of t_1 and t_2, the change cascades: $t_2[\text{Diagnosis}]$ also differs in D'. A second neighbor D'' with the same reassignment of $t_1[\text{Diagnosis}]$ but $t_2[\text{Diagnosis}]$ unchanged is equally valid—this captures the case where the dependency does not propagate, as when the RDR weight is low and the original pairing was already inconsistent. All cells outside $\mathcal{I}(c^*)$ remain identical in both cases. Fig. 2 compares this neighborhood relation to the standard DP notions.

4.3 Masking Privacy and Utility

We now formalize the mask pattern leakage channel.

Definition 4.3 (ε_m-Masking Privacy). *A randomized deletion mechanism $\mathbb{M}$ satisfies ε_m-*masking privacy *if for all neighboring databases $D \sim_{c^*} D'$ and all measurable sets of masks O:*

$$\Pr[\mathbb{M}(D, c^*) \in O] \leq e^{\varepsilon_m} \cdot \Pr[\mathbb{M}(D', c^*) \in O]. \tag{7}$$

To achieve masking privacy via the exponential mechanism [31], we define a utility function that balances deletion cost against re-inference leakage.

Definition 4.4 (Mask Utility). *For a mask $M \subseteq \mathcal{I}(c^*) \setminus \{c^*\}$, leakage threshold $L_0 \in (P, 1)$, and penalty parameter $\lambda > 0$:*

$$U(M, c^*, D) = -|M| - \lambda \cdot \mathbb{1}\left[\mathcal{L}(M, c^*, D) > L_0\right] \tag{8}$$

The first term penalizes larger masks by their size. The second imposes a penalty λ whenever leakage exceeds the threshold L_0 derived from the privacy budget (Eqn. 5). Higher utility indicates fewer deletions with leakage within budget. The parameter λ controls the relative importance of the leakage constraint versus deletion cost.

The exponential mechanism requires bounding the *sensitivity* of the utility function across neighboring databases [31]. Since neighbors may have different hypergraphs $\mathcal{H}_D^{c^*} \neq \mathcal{H}_{D'}^{c^*}$, the leakage $\mathcal{L}(M, c^*, D)$ can differ from $\mathcal{L}(M, c^*, D')$ for the same mask M. The indicator $\mathbb{1}[\mathcal{L}(M) > L_0]$ may therefore flip across neighbors: a mask that satisfies the threshold under one value of c^* may violate it under another.

Lemma 4.1. *For neighboring databases $D \sim_{c^*} D'$, the following holds: $\Delta u = \max_{M \subseteq \mathcal{I}(c^*) \setminus \{c^*\}} |U(M, c^*, D) - U(M, c^*, D')| \leq \lambda$.*

4.4 Composing Both Leakage Channels

We now show that ε_m-masking privacy (Defn. 4.3) and bounded re-inference leakage $\mathcal{L}(M) \leq L_0$ together imply the τ-Seal guarantee (Defn. 2.5). The proof proceeds in three steps. We decompose the pairwise likelihood ratio of the post-deletion observation into the two channels (Step 1), convert to a Bayes factor bound (Step 2), and translate to the success probability bound τ (Step 3).

Theorem 4.2. *If a randomized deletion mechanism $\mathbb{M}$ satisfies ε_m-masking privacy and produces a mask M with re-inference leakage $\mathcal{L}(M, c^*, D) \leq L_0$, then $\mathbb{M}$ satisfies τ-Seal with $\tau = \frac{e^{\varepsilon_m} \cdot L_0}{1 - L_0 + e^{\varepsilon_m} \cdot L_0}$, or equivalently,* $\mathrm{logit}(\tau) = \varepsilon_m + \mathrm{logit}(L_0)$.

Proof sketch. The post-deletion observation $D \setminus M$ encodes both the mask M and the visible cell values. By the chain rule, the pairwise likelihood ratio across neighbors $D \sim_{c^*} D'$ decomposes into a mask pattern factor (bounded by e^{ε_m} via masking privacy) and a re-inference factor (bounded by e^{ε_r} via the leakage constraint $\mathcal{L}(M) \leq L_0$), giving $\Pr[D \setminus M \mid c^*{=}v]/\Pr[D \setminus M \mid c^*{=}v'] \leq e^{\varepsilon_m + \varepsilon_r}$ for all values v, v'. This pairwise bound implies a Bayes factor bound: the odds update from prior to posterior satisfies $\mathrm{BF}(v) \leq e^{\varepsilon_m + \varepsilon_r}$ for any prior. Applying this to the true value $v^* = D[c^*]$ with prior probability at most P and converting from odds to probability yields the τ bound. Since $\varepsilon_r = \mathrm{logit}(\mathcal{L}) - \mathrm{logit}(P)$, the baseline log-odds cancel in the composition, yielding the form free of P. □

5 τ-SEAL-MECHANISMS

All our τ-Seal-mechanisms begin with constructing a hypergraph and use the constructed hypergraph to enumerate inference chains and compute leakage given a mask. Sec. 5.1 presents these algorithms. The τ-Seal mechanisms are in Sec. 5.2 and 5.3.

5.1 Hypergraph & Leakage Computation

Given a target cell $c^* = t_\ell[A_k]$ to be deleted, our mechanisms work with two local dependency hypergraphs around c^*. The *maximum* hypergraph $\mathcal{H}_{\max}^{c^*}$ conservatively includes all RDR instantiations that could involve c^* (independent of whether condition predicates hold under the current values), and is therefore stable across neighboring databases. We use $\mathcal{H}_{\max}^{c^*}$ to define the inference zone and the mask search space. The *actual* hypergraph $\mathcal{H}_D^{c^*}$ contains only those instantiations whose condition predicates are currently satisfied in D, and is therefore the structure on which leakage is evaluated.

Hypergraph construction. Both hypergraphs are constructed by frontier-based exploration from c^*. Starting with $V = \{c^*\}$, we repeatedly expand a frontier of newly discovered cells by instantiating only those RDRs that mention a frontier cell (in practice, RDRs are indexed by attribute so this lookup is constant-time per incident rule). Each instantiation yields one or more hyperedges, which we add to the edge set along with their rule weights. The search terminates when the frontier is empty, producing a local hypergraph $\mathcal{H} = (V, E)$. The inference zone is then $\mathcal{I}(c^*) = V(\mathcal{H}_{\max}^{c^*}) \setminus \{c^*\}$. By construction, $\mathcal{I}(c^*)$ depends only on the schema and RDRs (via $\mathcal{H}_{\max}^{c^*}$) and not on the current database values, keeping the candidate mask space fixed across neighboring databases. Due to space restrictions, we defer the algorithm to the full version of the paper.

Algorithm 1 Construct Local Hypergraph $\mathcal{H}^{c^*}$

Input: Database D, RDRs Σ, target cell c^*, mode $\in$ {Max, Actual}
Output: Hypergraph $\mathcal{H} = (V, E)$

```
V ← {c*}, E ← ∅; frontier ← {c*}, seen ← ∅
while frontier ≠ ∅ do
    next ← ∅
    for each c ∈ frontier with c ∉ seen do
        seen ← seen ∪ {c}
        for each σ ∈ IncidentRDRs(c, Σ) do
            edges ← Instantiate(σ, c, D, mode)
            for each hyperedge e in edges do
                E ← E ∪ {(e, w_σ)}; V ← V ∪ e
                next ← next ∪ (e \ seen)
    frontier ← next
return (V, E)
```

Chain enumeration. Alg. 2, given a hypergraph, enumerates the set of valid inference chains to c^* under mask M using a compact breadth-first exploration of the hypergraph. Lines 1–5 initialize the initially visible cells O, the output chain set $\mathcal{P}$, and two hash sets: Seen to de-duplicate completed chains by their ordered hyperedge sequence, and SeenSt to avoid revisiting the same search state (a partial chain together with its current set of known cells). Lines 6–7 define the activity predicate and canonical keys used throughout. Lines 9-12 first collect all length-1 chains, i.e., hyperedges that are directly active for inferring c^* from O, inserting each such chain into $\mathcal{P}$ only if its sequence key has not been seen before. Lines 14–19 initializes the BFS queue with all admissible first steps that infer a masked (non-target) cell x from the initial visibility set, recording the resulting state key to prevent redundant exploration. Lines 21–35 repeatedly pops a partial chain p with known set K and attempts to extend it by a new hyperedge e that shares at least one known witness cell (Line 24) and can infer a unique missing cell y given K (Line 25). The maximality condition of inference chains is enforced

Algorithm 2 EnumerateChains$(M, c^*, \mathcal{H})$

Input: Mask M, target c^*, hypergraph $\mathcal{H} = (V, E)$
Output: Valid chains $\mathcal{P}$ (unique by ordered hyperedge sequence)

```
O ← V\(M∪{c*}); P ← ∅; Q ← ∅; Seen ← ∅; SeenSt ← ∅
def Active(e, x, K) ≡ (x ∈ e) ∧ (e \ {x} ⊆ K)
def Seq(p) = ordered IDs of hyperedges in p;
def Key(p, K) = (Seq(p), fp(K))
for all e ∈ E with c* ∈ e do
    if Active(e, c*, O) and Seq((e)) ∉ Seen then
        P∪ = {(e)}; Seen∪ = {Seq((e))}
for all e ∈ E do
    for all x ∈ e \ (O ∪ {c*}) do
        if Active(e, x, O) then
            p ← (e); K ← O ∪ {x}
            if Key(p, K) ∉ SeenSt then Q.push(p, K);
            SeenSt∪ = {Key(p, K)}
while Q ≠ ∅ do
    (p, K) ← Q.pop()
    for all e ∈ E \ p do
        if e ∩ K = ∅ then continue
        pick any y ∈ e \ K with Active(e, y, K)        ▷ unique
        if no such y or Active(e, y, O) then continue
        p′ ← p ∘ e
        if y = c* then
            if Seq(p′) ∉ Seen then
                P ← P ∪ {p′}
                Seen ← Seen ∪ {Seq(p′)}
        else
            K′ ← K ∪ {y}
            if Key(p′, K′) ∉ SeenSt then
                Q.push(p′, K′)
                SeenSt ← SeenSt ∪ {Key(p′, K′)}
return P
```

by discarding extensions that would already be active under the initial visibility O (Line 26). If the inferred cell is the target, the resulting chain is added to $\mathcal{P}$ subject to sequence de-duplication (Lines 28–29); otherwise, the chain is enqueued with the updated knowledge state (Lines 30–33). The algorithm returns $\mathcal{P}$ in Line 36.

Leakage computation. Alg. 3 computes re-inference leakage for a given mask M on the induced hypergraph $\mathcal{H}_D^{c^*}$. The algorithm first enumerates all valid inference chains to c^* under M, computes each chain's weight and masked intermediate cells, then selects a weight-maximizing mask-disjoint collection via greedy selection, and finally aggregates success probabilities over the selected chains.

Lines 1–2 initialize the visible cells O and the chain set $\mathcal{P}$. The subroutine EnumerateChains (Line 3) performs a breadth-first exploration starting from O. Lines 4–7 compute each chain's weight $w(p)$ as the product of its hyperedge weights, and extract its masked intermediate cells $\mathsf{Masked}(p)$ (the witness cells belonging to M). Lines 8–14 perform greedy mask-disjoint selection: chains are processed in decreasing weight order (Line 9), and a chain is selected into $\mathcal{D}$ only if its masked cells do not overlap with any previously selected chain (Lines 11–13). Line 15 computes the final leakage as noisy-OR over the mask-disjoint collection $\mathcal{D}$.

Algorithm 3 ComputeLeakage$(M, c^*, \mathcal{H}_D^{c^*})$

Input: Mask M, target cell c^*, hypergraph $\mathcal{H} = (V, E)$
Output: Leakage $\mathcal{L}(M, c^*, D)$

```
O ← V \ (M ∪ {c*})                                  ▷ Visible cells
P ← EnumerateChains(M, c*, H)                       ▷ BFS from visible cells
for all p ∈ P do
    w(p) ← ∏_{e∈p} w_e                              ▷ Chain weight
    Masked(p) ← {x_i : i < |p|, x_i ∈ M}            ▷ intermediates
P_sorted ← SortByWeight(P, descending)
D ← ∅; used ← ∅
for all p ∈ P_sorted do
    if Masked(p) ∩ used = ∅ then
        D ← D ∪ {p}; used ← used ∪ Masked(p)
return 1 − ∏_{p∈D}(1 − w(p))                        ▷ Eqn. 6
```

The cost is dominated by chain enumeration and the greedy selection. Enumerating chains can be exponential in the worst case (since $|\mathcal{P}|$ may be large). Sorting chains takes $O(|\mathcal{P}| \log |\mathcal{P}|)$, and the greedy selection takes $O(|\mathcal{P}| \cdot |M|)$ using hash sets for the used cells. Thus, once $\mathcal{P}$ is materialized, leakage computation is $O(|\mathcal{P}| \log |\mathcal{P}| + |\mathcal{P}| \cdot |M|)$.

5.2 Exponential Mechanism for τ-Seal

First, we provide an exact implementation of τ-Seal using the exponential mechanism [31]. The full algorithm is deferred to the full version due to space limitations. The exponential mechanism samples an output from a discrete candidate set with probability proportional to the exponential of its utility, scaled by the privacy parameter and inversely scaled by the utility sensitivity. Here, the candidates are all deletion masks over the inference zone, and the utility trades off re-inference leakage against auxiliary deletions.

Alg. 4 presents the exponential deletion mechanism. It begins by constructing two hypergraphs. The maximum hypergraph $\mathcal{H}_{\max}^{c^*}$ (Line 1) includes all possible RDR instantiations regardless of whether condition predicates are satisfied, and determines the inference zone $\mathcal{I}(c^*)$ (Line 2). As discussed in Sec. 5.1, $\mathcal{H}_{\max}^{c^*}$ is value-agnostic and remains fixed across neighboring databases, ensuring that the candidate space is stable. The actual hypergraph $\mathcal{H}_D^{c^*}$ (Line 3) includes only valid instantiations whose condition predicates hold under the current database values, and captures which inference channels are currently active. Line 4 enumerates all subsets of the inference zone as candidate masks. Including all masks ensures the candidate space is identical across neighboring databases, which is essential for differential privacy. For each candidate mask $M \in \mathcal{R}$ (Lines 5–8), the algorithm computes leakage and utility. The leakage $\mathcal{L}(M)$ is computed via Alg. 3, which returns the tighter of inclusion–exclusion and noisy-OR estimates (Line 6). Utility $u(M)$ (Line 7) balances leakage suppression against deletion cost through Def. 4.4, where $\lambda \in (0, 1)$ controls the tradeoff: higher λ prioritizes low leakage, while lower λ prioritizes fewer deletions.

In practice, especially for retention-based deletions [11], cells of some attributes are frequently and periodically deleted. For such attributes, we use a two-step process: (1) **Offline template generation**: construct the value-agnostic maximum hypergraph and

Algorithm 4 Exponential Deletion Mechanism

Input: Database D, target cell c^*, RDRs Σ, parameters λ, ε
Output: Deletion mask M

Construct $\mathcal{H}_{\max}^{c^*}$ from Σ
Compute inference zone $\mathcal{I}(c^*) \leftarrow V(\mathcal{H}_{\max}^{c^*}) \setminus \{c^*\}$
Construct $\mathcal{H}_D^{c^*}$ from Σ and D (valid instantiations only)
Enumerate candidate masks $\mathcal{R} \leftarrow 2^{\mathcal{I}(c^*)}$ ▷ *All subsets*
for each $M \in \mathcal{R}$ **do**
  $\mathcal{L}(M) \leftarrow$ ComputeLeakage$(M, c^*, \mathcal{H}_D^{c^*})$
  $u(M) \leftarrow$ ComputeUtility$(M, \lambda, \mathcal{L}(M))$
Sample M with $\Pr[M] \propto \exp(\varepsilon \cdot u(M)/(2\lambda))$ ▷ $\Delta u = \lambda$)
return M

Algorithm 5 DelMarginal$(D, c^*, \mathcal{H}_{\max}^{c^*}, \varepsilon_m, L_0, \lambda)$

Input: Database D, target c^*, max hypergraph $\mathcal{H}_{\max}^{c^*}$, masking budget ε_m, leakage threshold L_0, penalty λ
Output: Mask $M \subseteq \mathcal{I}(c^*) \setminus \{c^*\}$

$\mathcal{Z} \leftarrow \mathcal{I}(c^*) \setminus \{c^*\}$, $M_0 \leftarrow \emptyset$
compute $\pi = (c_{\pi_1}, \ldots, c_{\pi_n})$ by sorting $\mathcal{Z}$ using Eqn. 9
for $k = 1, \ldots, n$ **do**
  $M_k \leftarrow M_{k-1} \cup \{c_{\pi_k}\}$
$\mathcal{H}_D^{c^*} \leftarrow$ ConstructHypergraph$(D, \Sigma, c^*, \text{Actual})$
for all $k \in \{0, \ldots, n\}$ **do**
  $\overline{\mathcal{L}}_k \leftarrow$ ComputeLeakage$(M_k, c^*, \mathcal{H}_D^{c^*})$
  $U_k \leftarrow$ ComputeUtility$(M_l, \lambda, \overline{\mathcal{L}}(M))$
sample $\hat{k}$ with $\Pr[\hat{k} = k] \propto \exp(\varepsilon_m U_k/(2\lambda))$
return $M_{\hat{k}}$

enumerate all masks offline, storing them as a template; (2) **Online mask sampling**: when a deletion request arises, compute utilities for candidate masks using the template and sample via the exponential mechanism.

5.3 Progressive Mask Generation for τ-Seal

We develop a mechanism based on the Gumbel-max trick [?] that constructs masks through a sequence of cell-level decisions while preserving τ-Seal. Given candidates with utilities $\{u_i\}$, drawing independent noise $g_i \sim \text{Gumbel}(0, b)$ and returning $\arg\max_i(u_i + g_i)$ is equivalent to sampling with probability proportional to $\exp(u_i/b)$. This avoids computing normalization constants and enables sequential decision-making.

Alg. 5 selects a mask M for deleting a target cell c^* by evaluating a conservative leakage bound on the actual induced hypergraph $\mathcal{H}_D^{c^*}$ and sampling from a fixed, nested candidate family of masks derived from the maximum hypergraph $\mathcal{H}_{\max}^{c^*}$. Let $\mathcal{Z} = \mathcal{I}(c^*) \setminus \{c^*\}$. We compute a static ordering $\pi = (c_{\pi_1}, \ldots, c_{\pi_n})$ over $\mathcal{Z}$ using $\mathcal{H}_{\max}^{c^*}$ (determined by the set of weighted RDRs and the schema) using the following weighted score

$$\text{score}(c) := \sum_{e \in E(\mathcal{H}_{\max}^{c^*}) : c \in e} \left(-\ln(1 - w_e)\right), \tag{9}$$

and form nested candidates $M_0 = \emptyset$ and $M_k = M_{k-1} \cup \{c_{\pi_k}\}$. For each k, we compute $\overline{\mathcal{L}}_k = \text{ComputeLeakage}(M_k, c^*, \mathcal{H}_D^{c^*})$ and compute utility as in Eq. 8. We sample $\hat{k}$ with $\Pr[\hat{k} = k] \propto \exp(\varepsilon_m U_k/(2\lambda))$ (Gumbel-max) and return $M_{\hat{k}}$.

Theorem 5.1. *Given a target cell c^* in a database D and a set Σ of weighted RDRs, let π be computed from $\mathcal{H}_{\max}^{c^*}$ such that it is identical on all neighboring databases. Then:*

- *Alg. 5 satisfies ε_m-masking privacy.*
- *For every candidate M_k, we have $\mathcal{L}(M_k, c^*, D) \le \overline{\mathcal{L}}(M_k, c^*, D)$. Hence, if $\overline{\mathcal{L}}(M_{\hat{k}}, c^*, D) \le L_0$ then $\mathcal{L}(M_{\hat{k}}, c^*, D) \le L_0$.*

Proof sketch. Since π is computed only from $\mathcal{H}_{\max}^{c^*}$, the candidate family $\{M_k\}_{k=0}^n$ is data-independent across neighbors. For any fixed k and neighbors $D \sim D'$, the utility $U_k(D) = -|M_k| - \lambda \cdot \mathbb{1}[\overline{\mathcal{L}}(M_k, c^*, D) > L_0]$ satisfies $|U_k(D) - U_k(D')| \le \lambda$, because $|M_k|$ is fixed and the indicator changes by at most one. Alg. 5 samples $\hat{k}$ with probability proportional to $\exp(\varepsilon_m U_k/(2\lambda))$, i.e., the exponential mechanism with sensitivity λ, which yields ε_m-masking privacy. ComputeLeakage returns an upper bound on the re-inference leakage. Applying this bound to each M_k gives $\mathcal{L}(M_k, c^*, D) \le \overline{\mathcal{L}}(M_k, c^*, D)$, which implies the threshold. □

6 EVALUATION

This section evaluates the effectiveness and efficiency of our two mechanisms that guarantee *Differential Inference Deletion (τ-Seal)*, namely, (i) **Exp**: the Exponential Mechanism (in §5.2) and (ii) **Gum**: the Progressive Mask Generation based on the Gumbel trick (in §5.3).

6.1 Experimental Setup & Overview

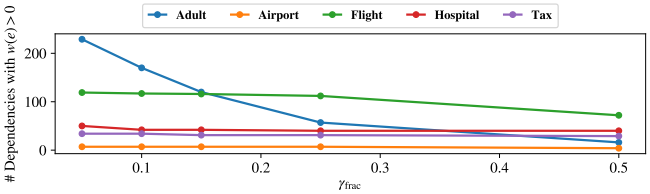
All experiments were run on an macOS Sequoia 15.1-based system (Apple M4 Pro; RAM: 24 GB). All algorithms are single-threaded, running on Python 3.13, and the datasets are stored in a MySQL (v9.4.0) database. The ILP approach uses the Gurobi (v13.0 Academic License) solver.

Datasets. We evaluate on four real-world datasets and one synthetic dataset (see Table 6c for a summary).

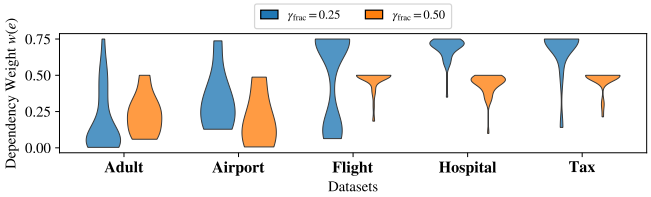
- Adult [25]: U.S. Census data with demographic and employment attributes. Constraints capture approximate socioeconomic correlations (e.g., age–education) and numeric comparisons, yielding a mix of moderate and strong inference paths.
- Airport [37]: Global airport records with geographic and operational attributes. Dependencies are largely structural, with near-deterministic rules induced by geographic hierarchies and identifiers.
- Hospital [5, 12]: Hospital data comprising facility descriptors and quality measures. Strong facility/geo dependencies coexist with weaker relationships on derived aggregates.
- Flight [25]: U.S. domestic flight records. Constraints are driven by flight identifiers, route consistency, and scheduling patterns, resulting in large inference neighborhoods.
- Tax [5, 12]: Synthetic taxpayer records with address and tax information, widely used to benchmark constraint-based data quality methods.

Metrics. We report the following metrics:

- **Mask-size improvement:** expected reduction in mask size relative to the result of **Min**: $100 \cdot (\mathbb{E}[|M_{\textbf{Min}}|] - \mathbb{E}[|M|])/\mathbb{E}[|M_{\textbf{Min}}|]$.



(a) Number of dependencies as a function of γ_{frac}.



(b) Weight distributions of dependencies.

Dataset	Dataset				MIN (SEAL [11])				EXP (τ-SEAL)						GUM (τ-SEAL)						T_{build}(EXP)
	#attr	#cells	RDR Source	$\lvert\Sigma\rvert$	T (ms)	Mem	η	$\mathcal{L}$	T (ms)	Mem	η	$\mathcal{L}$	M.I (%)	$\lvert\Pi\rvert$	T (ms)	Mem	η	$\mathcal{L}$	M.I (%)	$\lvert\Pi\rvert$	(ms)
AIRPORT	11	495K	[37]	7	10.9	1.1	0.20	0.00	1.5	3.1	0.23	0.11±0.02	22.5	3.1	3.6	0.5	0.27	0.11±0.02	32.5	2.7	5.59
HOSPITAL	15	1.7M	[5, 12]	40	21	2.9	0.10	0.00	3.4	54.7	0.12	0.32±0.01	45.6	43.8	19.0	1.8	0.17	0.30±0.02	59.9	46.4	597.57
ADULT	15	488K	[25]	57	36.0	5.6	0.10	0.00	2.0	231.9	0.15	0.21±0.02	51.1	180.0	61.4	2.9	0.27	0.20±0.02	72.0	214.2	8431.72
FLIGHT	20	10M	[25]	112	61.0	6.9	0.08	0.00	4.6	473.7	0.13	0.17±0.01	46.4	71.8	438.7	5.9	0.23	0.18±0.0	73.9	172.7	55680.97
TAX	14	1.4M	[5, 12]	31	7.5	0.6	0.25	0.00	2.3	0.6	0.75	0.22±0.02	33.3	0.7	3.4	0.2	0.28	0.32±0.03	36.7	0.7	4.57

(c) Dataset characteristics and deletion overheads over 100 deletion requests with $(\epsilon_m = 0.1, L_0 = 0.2)$.
$|\Sigma|$: Set of RDR. T : Total time for deletion (online phase only for EXP). Mem : Memory of model. η : Efficiency. $\mathcal{L}$: Re-inference leakage. M.I (%): Mask Improvement. $|\Pi|$: Channel complexity (# inference chains after masking). T_{build} : Time to build template in the offline phase of EXP.

Figure 6: Assigning weights to dependencies using confidence-bounded gain beyond a base rate.

This metric normalizes savings across datasets and highlights how much overhead is avoided by tolerating leakage.

- **Inferential leakage ($\mathcal{L}(M)$):** probability of re-inference after masking cells in the mask M, reported as a percentage.
- **Deletion ratio:** fraction of the inference zone that is masked, $|M|/|I(c^*) \setminus \{c^*\}|$, enabling meaningful cross-dataset comparison.
- **Efficiency (η):** a joint summary of deletion utility and leakage, $\eta = (1 - \mathcal{L})/(|M| + 1)$. This captures the effective re-inference risk-reduction per deleted cell.
- **Time:** end-to-end latency T (ms) per deletion request and preprocessing cost (reported as offline build time T_{build} for **EXP** when applicable), and when relevant, a decomposition into initialization, model construction, and update phases.
- **Memory (`Mem`):** size of the mechanism's materialized model, thus capturing the memory cost of enabling secure deletions.
- **Channel complexity ($|\Pi|$):** number of distinct inference channels induced for the target under the leakage model. This characterizes the structural difficulty of the instance and helps explain high-leakage or high-overhead regimes.

Baselines. Three deterministic inference-blocking strategies exist [11]: those that delete all dependent data [3, 13]; delete one cell per dependency [34]; and, **MIN** [11], which minimizes deletions via ILP. By design, **MIN** incurs the lowest auxiliary deletions. So we primarily compare against **MIN**. We follow [11] and evaluate one-hundred deletions using the three mechanisms.

6.2 Overview

After explaining how we assign weights to dependencies in Sec. 6.3, our evaluation is organized into three parts. To establish the leakage–deletion tradeoff and its sensitivity to the re-inference budget L_0 and the mask-privacy budget ϵ_m, we vary both parameters across all datasets and measure auxiliary deletions and leakage (Sec. 6.4). To compare **EXP** and **GUM**, we profile mask quality, runtime, and memory under identical deletion workloads (Sec. 6.5). To understand how the total privacy budget τ should be allocated between the two channels, we sweep over splits of τ into ϵ_m and L_0 at fixed τ and measure mask improvement (Sec. 6.6). Unless stated otherwise, each reported point aggregates 100 deletion requests per dataset at the default parameter setting ϵ_m=0.1, L_0=0.2, λ=1000 ($\tau \approx 0.22$). Table 6c reports all metrics at this setting.

We observe the following. First, both **EXP** and **GUM** consistently reduce auxiliary deletions relative to **MIN** across all datasets, with the re-inference budget L_0 being a reliable parameter for controlling the leakage–auxiliary deletion tradeoff (Sec. 6.4). Second, the two mechanisms complement each other in the runtime—memory space: **EXP** achieves 3–14× faster online deletions than **MIN** by shifting work to an offline template-build phase, while **GUM** avoids enumeration and maintains a memory overhead 30–80× smaller than **EXP**, at the cost of higher per-request latency on densely connected inference zones. Third, the split of the privacy budget τ into ϵ_m and L_0, as expected, starkly affects mask-size improvement. On datasets with small inference zones and a high number of dependencies, the gap between the best and worst splits reaches up to 81 percentage points, and the optimal allocation is predictable from the complexity of the inference zone.

This longer version of the paper contains comprehensive plots for all the datasets and all the experiments.

6.3 Assigning Weights to Dependencies

We assign weights to dependencies that reflect how reliably they enable *correct inference*. Rather than treating every discovered denial constraint (DC) as a deterministic inference channel (weight 1), we assign each DC a confidence-bounded weight $w(e) \in [0, 1]$ that quantifies its *predictive lift beyond a base rate*.

For each dependency, $e \equiv \neg(\varphi_1 \wedge \cdots \wedge \varphi_m)$ over at most two tuple variables (t_0, t_1) with $m \leq 3$, we view each predicate φ_j as a possible *head* and treat the remaining predicates $E_{e,j} = \bigwedge_{i \neq j} \varphi_i$ as the

premise. The DC implies that whenever $E_{e,j}$ holds, the head predicate cannot also hold; hence the head-specific inference event is $\neg\varphi_j$. We estimate these quantities on $S(d)$ uniformly sampled tuple-pairs for each dataset d. Among valid pairs V_e (all referenced attributes which are non-null), we define the baseline $p_0(e,j) = \Pr[\neg\varphi_j \mid V_e]$ and the conditional accuracy $p_1(e,j) = \Pr[\neg\varphi_j \mid E_{e,j}]$. To make the test robust to sampling noise, we compute confidence bounds and define a residual-scaled margin $\gamma(e,j) = \gamma_{\text{frac}} \cdot (1 - \text{UCB}(p_0(e,j)))$. The directional weight is then the confidence-bounded normalized improvement:

$$w(e,j) = \max\left\{0,\ \frac{\text{LCB}(p_1(e,j)) - \text{UCB}(p_0(e,j)) - \gamma(e,j)}{1 - \text{UCB}(p_0(e,j))}\right\},$$

and the DC weight is $w(e) = \max_j w(e,j)$. Intuitively, $w(e)$ measures the fraction of *residual uncertainty* (beyond the base rate) removed by the DC, *with confidence* γ_{frac}. To avoid unstable weights from tiny support, we enforce minimum evidence thresholds on both the validity set and the premise: we require $|V_e| \geq n_V$ and $|E_{e,j}| \geq n_E(d)$. We fix n_V globally, while $n_E(d)$ is dataset-scaled; this guards against DCs with premises that are rarely true while still allowing moderately sized datasets to retain meaningful DCs.

Fig. 6a shows the intended monotone effect of the reliability margin: as γ_{frac} increases, fewer DCs retain positive weight, since the confidence-bounded gain test becomes stricter and more directions fail to clear the residual-scaled margin. For the two settings we use in the evaluation, Fig. 6c shows the number of dependencies retained for each dataset. Tightening from $\gamma_{\text{frac}} = 0.25$ to 0.50 prunes the DC set substantially for ADULT (and also noticeably for FLIGHTS), while AIRPORT and TAX shrink more mildly, and HOSPITAL remains essentially unchanged, indicating that the DCs that survive at $\gamma_{\text{frac}} = 0.25$ in HOSPITAL already clear a stricter margin.

Fig. 6b highlights that the *weight spectrum* is dataset-dependent even after the same confidence-bounded normalization. Across datasets, most retained DCs have relatively small positive weights (i.e., modest but statistically reliable lift beyond the base rate), with ADULT and AIRPORT concentrating mass near the low end, while FLIGHTS exhibits a broader tail of positive weights. Importantly, we do not introduce an additional τ cutoff: for each γ_{frac} we retain exactly the DCs with $w(e) > 0$, so these plots directly characterize the dependency sets used by the evaluation.

6.4 Leakage and Deletion Tradeoff

We first establish the tradeoff between inferential leakage and auxiliary deletions by studying how the two main parameters, i.e., the re-inference budget L_0 and the mask-privacy budget ϵ_m, govern it.

Exp. 1: Sensitivity to L_0 and ϵ_m. Fig. 7 and Fig. 8 plot, for each dataset and each mechanism, mean mask-size improvement over **MIN** (top row) and mean realized re-inference leakage (bottom row) as L_0 varies from 0.1 to 0.9. Each curve corresponds to a value of $\epsilon_m \in \{0.1, 0.5, \ldots, 10.0\}$.

Across all datasets, increasing L_0 improves mask size, but the shape of the improvement curve is dataset-dependent. On HOSPITAL, the curve rises steadily: relaxing L_0 from 0.1 to 0.6 lifts improvement from 8% to 43%, while leakage grows from 2% to 44%. Contrastingly, on AIRPORT, the curve plateaus early: improvement reaches ≈40% by L_0=0.3 and changes little thereafter. This indicates that sparse inference neighbourhoods have few paths to suppress, so modest leakage tolerance already unlocks most savings; further relaxation primarily increases leakage without shrinking the mask.

The effect of ϵ_m differs between the two mechanisms. For **EXP** (Fig. 7), larger ϵ_m shifts each curve upward: on ADULT, improvement rises from 35% at ϵ_m=0.1 to 89% at ϵ_m=10, while leakage grows only from 13% to 27%. At the same time, the standard deviation of the mask reduces, because a larger ϵ_m permits the exponential mechanism to concentrate more probability on high-utility masks, at the cost of a weaker mask-pattern privacy guarantee.

On the other hand, as expected, **GUM** (Fig. 8), behaves differently: on several datasets improvements can *decrease* with larger ϵ_m, and variance remains high. This reflects the fact that ϵ_m couples directly with the Gumbel noise scale in the sequential construction, amplifying randomness rather than concentrating it. In practice, $\epsilon_m \in [0.1, 1.0]$ is the useful range for both mechanisms; beyond ϵ_m≈2, **EXP** saturates and **GUM** becomes erratic.

Exp. 2: Default operating point. Having mapped the parameter landscape, we fix a conservative operating point (ϵ_m=0.1, L_0=0.2, λ=1000, giving τ≈0.22) and report all metrics in Fig. 6c. The table lists, for each dataset and mechanism, the mean mask size $|M|$, mask-size improvement (%), realized leakage $\mathcal{L}$, deletion ratio $|M|/|I(c^*)|$, efficiency η, end-to-end time T, memory, and channel complexity $|\Pi|$.

Even at this conservative setting, both mechanisms reduce auxiliary deletions on every dataset except TAX, where the inference zone already equals $|M_{\text{MIN}}|$=3 and no reduction is possible. The reductions grow with inference-zone richness (i.e., number of constraints): on FLIGHT, **GUM** drops $|M|$ from 11.0 to 3.1 (72% improvement at 17% leakage) and **EXP** to 5.6 (49% at 15%). On ADULT, **GUM** achieves 55% improvement at 8% leakage versus **EXP**'s 43% at 11%. On AIRPORT, the gap is narrower: 10–14% improvement at 5–6% leakage which i consistent with the early saturation observed in Exp. 1. The HOSPITAL dataset shows moderate improvement (5–17%) despite its large inference zone ($|I(c^*)|$=10), because its many high-weight dependencies make leakage expensive to suppress.

Exp. 3: Leakage—deletion frontier. Fig. 9 plots mean mask size $|M|$ (y-axis) against mean $\mathcal{L}(M)$ (x-axis) for each dataset, sweeping L_0 at two mask-pattern budgets $\epsilon_m \in \{0.1, 1.0\}$. **MIN** appears as a single point fixed at $\mathcal{L}$=0 with $|M|$=$|M_{\text{MIN}}|$.

Both **EXP** and **GUM** exhibit a descending, concave frontier: initial leakage tolerance yields large mask reductions, but returns diminish as the curves flatten. The location of the "knee" is dataset-dependent. AIRPORT saturates by L_0=0.3 capturing nearly all improvement is at a small leakage. HOSPITAL and ADULT have a longer useful range with knees around L_0=0.5, and TAX is nearly vertical, reflecting its all-or-nothing structure.

The two ϵ_m settings separate visibly on dense datasets. On ADULT and FLIGHT, the ϵ_m=1.0 curves lie well below ϵ_m=0.1: allocating more budget to mask-pattern privacy unlocks smaller masks at the same leakage level. On AIRPORT and TAX, the two settings nearly overlap, confirming that sparse structures gain little from additional mask-pattern budget. We will analyze this gap by considering various budget-splitting scenarios in Sec. 6.6, where we quantify the mask-size difference between the best and worst (ϵ_m, L_0) splits for a given τ.

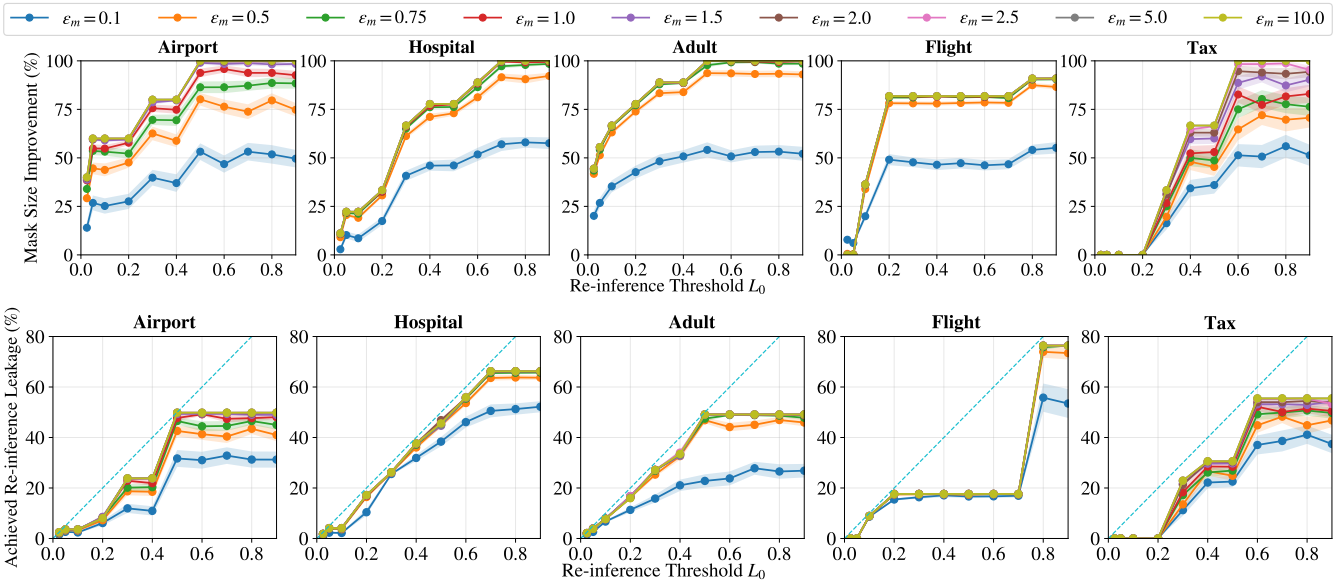


Figure 7: Sensitivity of Exp to parameters: ϵ_m and L_0. Bottom row shows achieved re-inference leakage.

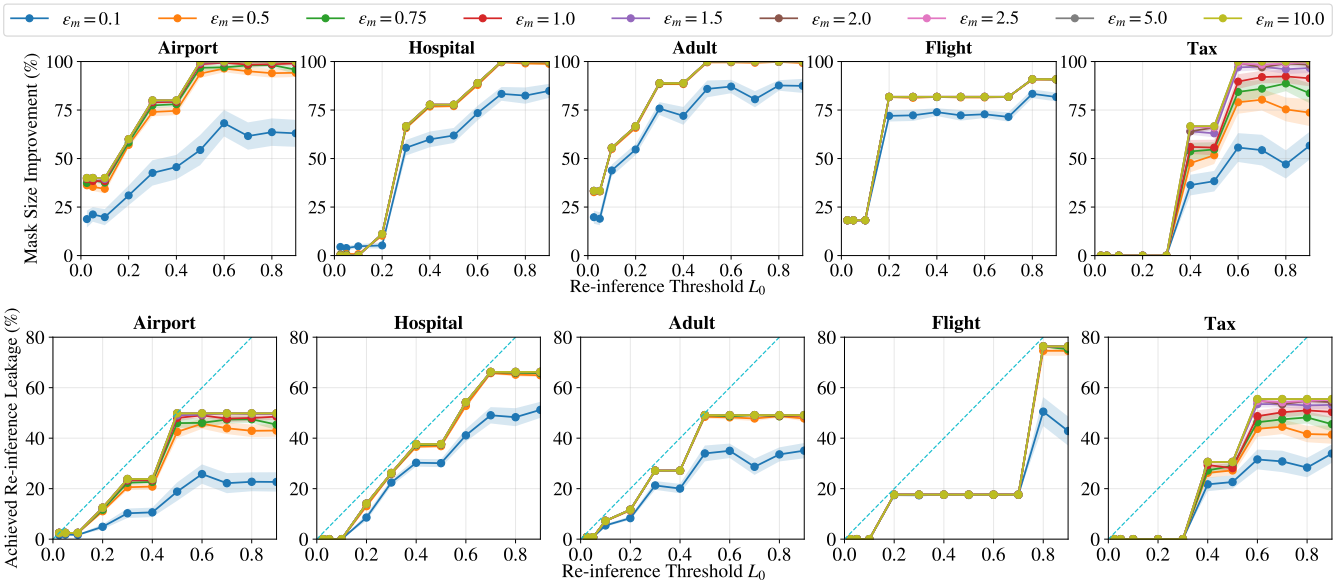


Figure 8: Sensitivity of Gum to parameters: ϵ_m and L_0. Bottom row shows achieved re-inference leakage.

6.5 Comparing Mechanisms

We now compare **Exp** and **Gum** along four axes: mask quality, runtime, memory, and sensitivity to the dependency structure of the inference zone.

Exp. 4: Mask quality. Fig. 6c reports, for each dataset, the mean mask size, improvement over **Min**, realized leakage, deletion ratio, and efficiency η at the default operating point. We compare the two mechanisms row by row.

On datasets with large inference zones, **Gum** consistently produces smaller masks than **Exp**. On Flight ($|I(c^*)|$=13), **Gum** achieves 72% improvement versus **Exp**'s 49%; on Adult ($|I(c^*)|$=12), the gap is 55% versus 43%. The reason is structural: **Gum**'s progressive construction builds masks by greedily adding cells in order of their leakage contribution, which is effective when the inference zone is large and many cells contribute marginally. **Exp**, by contrast, samples from the full power set weighted by utility, and its sampling

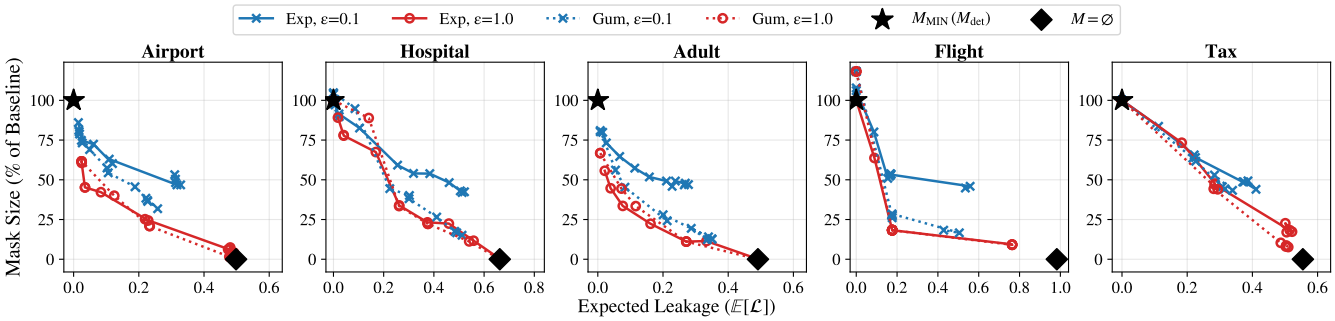


Figure 9: Pareto curves.

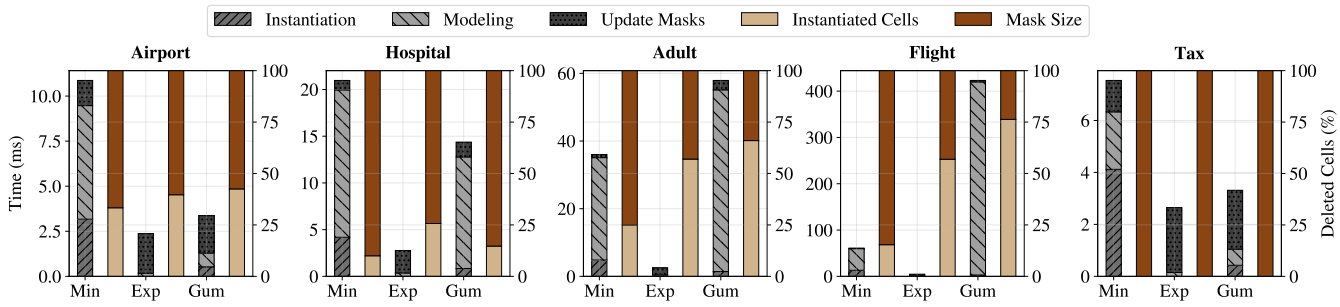


Figure 10: For each dataset: runtime, number of cells instantiated and deleted.

noise floor prevents it from concentrating as aggressively on the smallest feasible mask at low ϵ_m.

The pattern reverses on Hospital, where **Exp** outperforms **Gum** (17% vs. 5%). Here the inference zone is moderately sized ($|I(c^*)|$=10) but densely connected: many high-weight edges create strong coupling between cells, so **Gum**'s greedy ordering—which is fixed by the maximum hypergraph and does not adapt to the actual hypergraph—misses masks that **Exp**'s global sampling can find.

On Airport ($|I(c^*)|$=6), both mechanisms achieve comparable improvement (10–14%), and on Tax ($|I(c^*)|$=3), both match **Min** exactly—the zone is too small for any reduction.

The efficiency metric $\eta = (1-\mathcal{L})/(|M|+1)$ summarises this comparison. **Gum** attains the highest η on Flight (0.22, nearly tripling **Min**'s 0.08) and Adult (0.19 vs. **Exp**'s 0.15). **Exp** leads on Hospital (0.17 vs. 0.09). The leakage variance tells a complementary story: **Exp** exhibits lower standard deviation in realized leakage across the 100 deletion requests on every dataset, making it the more predictable mechanism when stable worst-case guarantees matter.

Exp. 5: Runtime. Fig. 10 decomposes end-to-end deletion time into three phases: instantiation (constructing the hypergraph from the database), modelling (evaluating leakage for candidate masks), and the update step (applying the selected mask). Each bar shows the mean over 100 deletion requests.

The two mechanisms embody a classic offline/online tradeoff. **Exp** completes each online deletion in 2–5 ms regardless of dataset, because the expensive work—enumerating all candidate masks and computing their utilities—is performed once during an offline template-build phase. **Gum** has no offline phase but pays per request: 3.4 ms on Airport, 58 ms on Adult, and 423 ms on Flight. The modelling phase dominates **Gum**'s cost on densely connected inference zones, because each candidate in the nested family requires a fresh leakage evaluation via chain enumeration. Compared to **Min**, which solves an ILP per request (10.9–61.0 ms), **Exp**'s online phase is 4–15× faster, and **Gum** is faster on four of five datasets—the exception being Flight.

Offline template build. **Exp**'s offline phase constructs the maximum hypergraph, enumerates the full candidate mask space $2^{I(c^*)\setminus\{c^*\}}$, and precomputes utility scores for each candidate. The cost scales with $|I(c^*)|$: T_{build} is negligible on Airport and Tax (both < 0.1 s), moderate on Hospital (0.5 s), and substantial on Adult (7.3 s) and Flight (48.5 s). On Flight, the inference zone contains 13 cells, yielding 2^{12}=4096 candidate masks, each requiring chain enumeration. This cost is paid once per target attribute and can be amortised across deletion requests for the same attribute—a common pattern in retention-driven erasure (§??).

Exp. 6: Memory. Fig. 6c also reports the memory footprint of each mechanism's materialised model. **Exp** stores the full candidate set with precomputed utilities: ≈474 KB on Flight, 232 KB on Adult, and < 5 KB on Airport and Tax. **Gum** stores only the static cell ordering over the inference zone: ≈5.9 KB on Flight, 2.9 KB on Adult—a reduction of approximately 80×. **Min** materialises the ILP model per request and discards it, so its persistent memory cost is near zero but its per-request allocation is higher.

The memory–runtime tradeoff is clear: **Exp** trades storage for fast online deletions, while **Gum** trades compute for a small footprint. In settings where deletion requests arrive in bursts for the same target attribute (e.g., retention-driven erasure), **Exp**'s offline investment pays off. When deletions are ad-hoc across different attributes, **Gum**'s negligible memory and zero build cost make it the practical choice.

Exp. 7: Structural sensitivity. The dependency structure of the inference zone—captured by the channel complexity $|\Pi|$ reported in Fig. 6c—mediates both leakage regimes and runtime behaviour.

Fig. 7 and Fig. 8 show that Flight incurs the highest realised leakage across all L_0 settings, while Airport and Tax remain at low leakage even as L_0 grows. At the default operating point, **Exp** attains $\mathcal{L}\approx 0.15$ on Flight versus $\mathcal{L}\approx 0.06$ on Airport at comparable mask reduction. The explanation is that densely connected inference zones maintain many simultaneously active inference channels under partial masking, so each cell removed from the mask re-exposes multiple paths to c^*.

The deletion ratio provides a complementary view. **Min** masks most of the inference zone across all datasets (ratio $\in [0.67, 1.00]$). **Gum** is particularly aggressive: on Flight, the ratio drops from 0.85 (**Min**) to 0.24—masking only a quarter of the zone while bounding leakage at 17%.

The dependency structure of the inference zone also explains where **Gum**'s runtime becomes expensive. In densely connected inference zones, proposed cell removals are frequently near the L_0 threshold, triggering costly leakage re-evaluations at each step of the progressive construction. This is why **Gum** is a runtime outlier on Flight (423 ms vs. 4.7 ms for **Exp**) but comparable on sparse datasets like Airport (3.4 ms vs. 2.4 ms).

6.6 Splitting the Privacy Budget

Different (ϵ_m, L_0) pairs yield the *same* guarantee (τ) but allocate the privacy budget differently between mask-pattern privacy and re-inference suppression. A natural question is: given a fixed τ, which split produces the smallest masks in practice?

Exp. 8: Empirical budget splitting. Fig. 11 shows mask-size improvement over **Min** (%) across the full (ϵ_m, L_0) space as a heatmap, with iso-τ contours overlaid. Each cell in the heatmap reports the mean improvement over 100 deletion requests for that parameter combination.

The key observation is that the improvement landscape is *not* aligned with the iso-τ contours: parameter combinations providing identical privacy guarantees show markedly different improvements. On Flight (**Exp**), the $\tau\approx 0.32$ contour spans improvements from 31% to 81% depending on the split. On Adult, the same contour spans 44% to 78%. This means that two deployments offering the same adversarial bound τ can differ by up to 50% in how much data they preserve, purely based on how the budget is split.

Fig. 12 quantifies this gap at four representative values of τ $\{0.15,\ 0.32,\ 0.52,\ 0.73\}$, plotting the best- and worst-split improvement for each dataset and mechanism. On Flight at $\tau\approx 0.32$, the best split (ϵ_m=0.75, $L_0\approx 0.20$) achieves 81% improvement over **Min**, while the worst (ϵ_m=0.1, L_0=0.30) achieves only 49%—a gap of 32% for **Exp**. For **Gum** on the same dataset and τ, the gap widens further: the worst split recovers essentially no improvement over **Min** while the best achieves over 70%. The effect grows with τ: at $\tau\approx 0.73$, Hospital shows a gap of over 40% for both mechanisms.

Exp. 9: Optimal split and heuristic. The optimal ϵ_m^* follows a clear pattern tied to the dependency structure of the inference zone. For sparsely connected zones (Airport: mean $|\Pi|\approx 4$; Tax: $|\Pi|\leq 1$), ϵ_m^* is higher (1.0–2.0)—few inference paths to suppress, so the dominant threat is mask-pattern leakage and budget should flow to ϵ_m. For densely connected zones (Hospital: $|\Pi|\approx 75$; Adult: $|\Pi|\approx 130$; Flight: $|\Pi|\approx 98$), ϵ_m^* is moderate (0.5–1.5)—many active inference channels make re-inference the dominant threat and budget should flow to L_0.

This suggests a simple heuristic for practitioners: estimate $|\Pi|$ from the dependency structure of the inference zone and favour ϵ_m when $|\Pi|$ is small, L_0 when $|\Pi|$ is large.

Summary. Our evaluation affirms that meaningful data deletion need not be all-or-nothing. By tolerating bounded leakage, our mechanisms reduce auxiliary deletions relative to the deterministic baseline **Min**, while providing formal τ-Seal guarantee. **Exp** and **Gum** are complementary: the former is suitable when low online latency is critical and memory for precomputed templates is available, whereas the latter suits memory-constrained settings or schemas with complex inference zones where full enumeration is impractical. Across all five datasets, the re-inference budget L_0 emerges as the most intuitive control knob for practitioners: increasing L_0 yields a smooth, predictable reduction in auxiliary deletions, and the budget split between ϵ_m and L_0 can be guided by the structural complexity of the inference zone. These results demonstrate that the τ-Seal framework, yields a practical middle ground between the excessive deletions of deterministic approaches and the absence of formal guarantees in ad-hoc strategies, with L_0 and ϵ_m serving as interpretable control knobs for navigating this space.

7 RELATED WORK

The problem of probabilistic privacy-preserving data deletion sits at the intersection of several research areas, each addressing complementary but insufficient aspects of the challenge.

Deletion propagation and meaningful erasure. Classical view-update and deletion propagation frameworks [7, 23] study how deletions cascade under logical dependencies. ILP-based propagation [30] seeks to minimize side effects on source data and derived views, but does not address inference prevention and often incurs high overheads: time and additional deletions. Graph-based deletion methods [3, 13] ensure correctness via ownership-based cascades but typically over-delete due to coarse dependency abstractions. P2E2 [11] formalizes meaningful erasure as preserving insertion-time inferability and uses deterministic mechanisms over RDRs where inference is either fully enabled or fully blocked —two extremes. Unlike these works, *we consider a probabilistic inference framework* with weighted dependencies, and probabilistic guarantees on leakage due to deletion patterns.

Differential privacy (DP). Classical DP [14] defines indistinguishability under a tuple-level neighboring relation whose semantics become subtle when data exhibits correlations or constraints. This has motivated semantic generalizations such as Pufferfish [22] and Blowfish [18], which specify secrets and correlations through explicit world models or policy graphs, and Dependent Differential

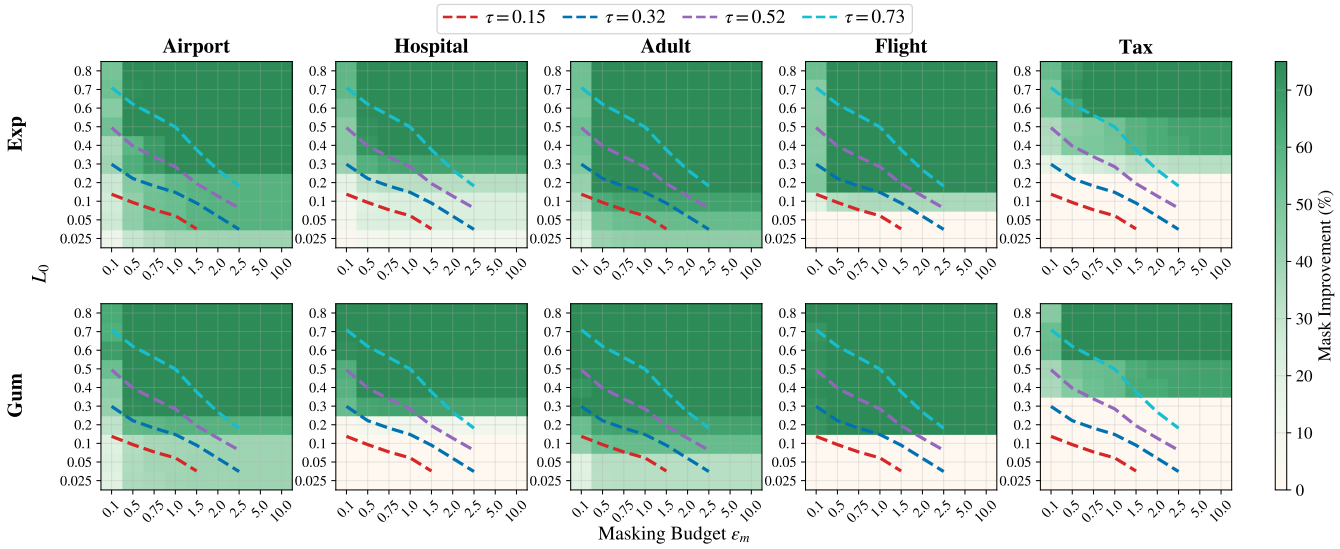


Figure 11: Mask-size improvement (%) over Min across the (ϵ_m, L_0) parameter space. Top row: Exp; bottom row: Gum. Dashed curves are iso-τ contours. The improvement landscape is not aligned with the contours: identical τ values yield markedly different improvements depending on the budget split.

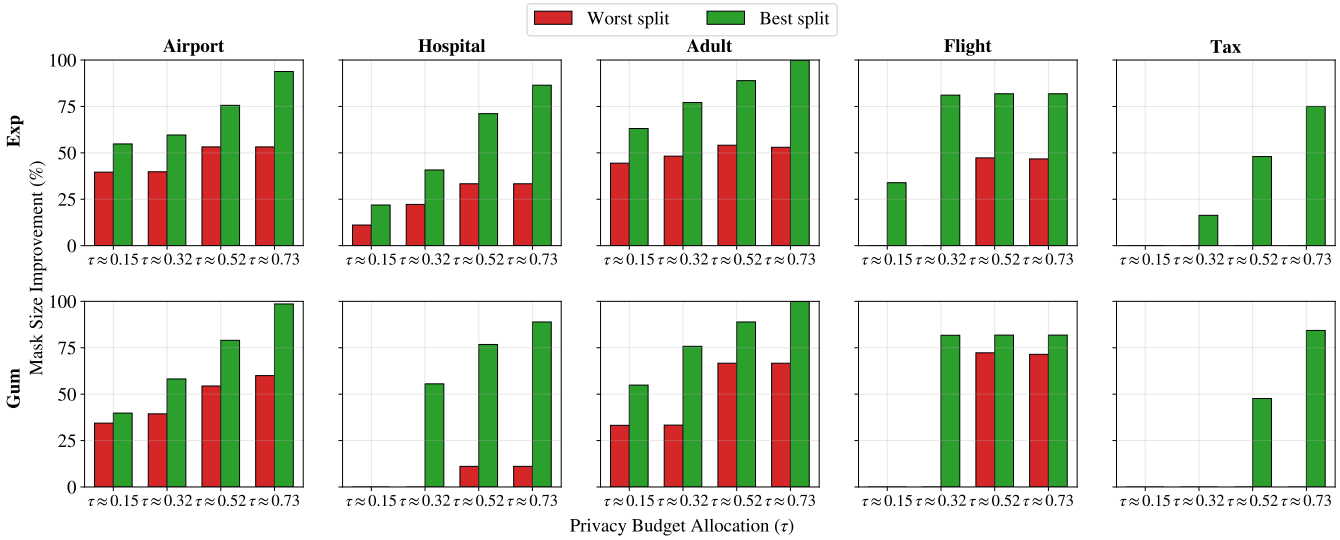


Figure 12: Best- vs. worst-split mask-size improvement (%) over Min at four representative τ levels. Left group in each pair: Exp; right group: Gum. The gap between best and worst split grows with τ and is largest on densely connected datasets.

Privacy (DDP) [28], which quantifies privacy loss under correlated data via dependence coefficients. These differ from dependencies considered in our framework. DP systems for SQL engines [19, 24], federated querying [4], streaming [38], update-pattern hiding [40], and inconsistency estimation [32], add noise to query outputs or access patterns. Instead, *we ensure that the deletion mechanism produces a mask that preserves the privacy of the deleted value.*

Machine unlearning. Machine unlearning aims to remove the influence of training data from learned models through retraining, gradient adjustments, or data removal [8, 16, 17]. These guarantees apply to model parameters. Our work, in contrast, is at the data layer where dependencies may exist between the input and output of a trained model. The two approaches are complementary: *our deletion mechanisms prevent leakage through the visible database state*, while machine-unlearning prevents leakage of training data (requested to be deleted) through trained models.

8 CONCLUSION & FUTURE WORK

We introduced τ-Seal, a probabilistic relaxation of deterministic meaningful deletion framework that bounds an adversary's re-inference probability via two composable leakage channels: re-inference leakage and masking privacy. We designed two complementary mechanisms that guarantee τ-Seal while reducing auxiliary deletions. We identify several directions for adoption.

Prior deterministic approaches treat dependencies as binary: a rule is either active and must be blocked, or discarded entirely. This creates two problems. Weak dependencies that clear the threshold incur the same deletion cost as strong ones, leading to excessive deletions. Conversely, dependencies below the threshold are assumed to contribute zero leakage, yet even individually weak channels compose through the noisy-OR aggregation (Eqn. 6), so discarding them understates the adversary's true re-inference probability.

τ-Seal addresses both the issues. Continuous weights let our mechanisms concentrate deletions on high-weight paths while tolerating controlled leakage through weak ones. Moreover, by considering the baseline probability P (the adversary's success when *all* dependent data is masked) our framework measures the *advantage* gained from the post-deletion database rather than treating inference as all-or-nothing. Existing deterministic frameworks have no analogue of P: they block all paths without quantifying how much the adversary's odds actually shift.

As in all dependency-based prior work, we require the set of dependencies be specified upfront, a requirement aligned with the "reasonableness" principle in compliance domains. A formalization bridging legal standards of reasonable re-inference risk and the quantitative guarantees offered by τ remains an important direction.

When deletion requests have overlapping inference zones (in workloads with recurring erasure obligations) formalizing how the τ-Seal guarantee composes beyond standard DP composition bounds remains open.

Dependency weights are currently profiled from a static snapshot. As the database evolves, insertions may widen inference zones and updates may strengthen previously benign paths, both potentially violating τ-Seal satisfied at deletion time. In such cases, we can fall back to conservative over deletion to preserve the privacy guarantee at the cost of forgoing savings. Avoiding this fallback at scale may be possible by pairing incremental dependency discovery [37] with a per-request leakage monitor that triggers targeted mask repair only when a materialized change pushes the running estimate beyond τ.

REFERENCES

[1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting data errors: where are we and what needs to be done? *PVLDB* 9, 12 (Aug. 2016), 993–1004.

[2] Archita Agarwal, Marilyn George, Aaron Jeyaraj, and Malte Schwarzkopf. 2021. Retrofitting GDPR compliance onto legacy databases. *PVLDB* 15, 4 (Dec. 2021), 958–970.

[3] Kinan Dak Albab, Ishan Sharma, Justus Adam, Benjamin Kilimnik, Aaron Jeyaraj, Raj Paul, Artem Agvanian, Leonhard Spiegelberg, and Malte Schwarzkopf. 2023. K9db: Privacy-Compliant Storage for Web Applications by Construction. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX, Boston, MA, USA, 99–116.

[4] Johes Bater, Xi He, William Ehrich, Ashwin Machanavajjhala, and Jennie Rogers. 2018. Shrinkwrap: efficient SQL query processing in differentially private data federations. *PVLDB* 12, 3 (Nov. 2018), 307–320.

[5] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient denial constraint discovery with hydra. *PVLDB* 11, 3 (Nov. 2017), 311–323.

[6] Loreto Bravo, Wenfei Fan, and Shuai Ma. 2007. Extending dependencies with conditions. In *Proceedings of the International Conference on Very Large Databases (VLDB)*. VLDB Endowment, 243–254.

[7] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2002. On propagation of deletions and annotations through views. In *Proceedings of the Symposium on Principles of Database Systems (PODS)*. Association for Computing Machinery, Madison, Wisconsin, 150–158.

[8] Yinzhi Cao and Junfeng Yang. 2015. Towards Making Systems Forget with Machine Unlearning. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*. IEEE Computer Society, Washington, DC, United States, 463–480.

[9] Loredana Caruccio, Vincenzo Deufemia, Felix Naumann, and Giuseppe Polese. 2021. Discovering Relaxed Functional Dependencies Based on Multi-Attribute Dominance. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 33, 9 (2021), 3212–3228.

[10] Vishal Chakraborty, Stacy Ann-Elvy, Sharad Mehrotra, Faisal Nawab, Mohammad Sadoghi, Shantanu Sharma, Nalini Venkatsubhramanian, and Farhan Saeed. 2024. Data-CASE: Grounding Data Regulations for Compliant Data Processing Systems. In *Proceedings of the International Conference on Extending Database Technology (EDBT)*. OpenProceedings, Paestum, Italy, 108–115.

[11] Vishal Chakraborty, Youri Kaminsky, Sharad Mehrotra, Felix Naumann, Faisal Nawab, Primal Pappachan, Mohammad Sadoghi, and Nalini Venkatasubramanian. 2025. Meaningful Data Erasure in the Presence of Dependencies. *PVLDB* 18, 10 (2025), 3435 – 3448. https://doi.org/10.14778/3748191.3748206

[12] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *PVLDB* 6, 13 (Aug. 2013), 1498–1509.

[13] Katriel Cohn-Gordon, Georgios Damaskinos, Divino Neto, Joshi Cordova, Benoît Reitz, Benjamin Strahs, Daniel Obenshain, Paul Pearce, and Ioannis Papagiannis. 2020. DELF: Safeguarding Deletion Correctness in Online Social Networks. In *Proceedings of the USENIX Conference on Security Symposium*. USENIX, USA, Article 60, 18 pages.

[14] Cynthia Dwork and Aaron Roth. 2014. The Algorithmic Foundations of Differential Privacy. *Foundations and Trends in Theoretical Computer Science* 9, 3–4 (2014), 211–407. https://doi.org/10.1561/0400000042

[15] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional Functional Dependencies for Capturing Data Inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2, Article 6 (2008), 48 pages.

[16] Antonio A. Ginart, Melody Y. Guan, Gregory Valiant, and James Zou. 2019. Making AI forget you: data deletion in machine learning. In *Proceedings of the International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, Article 316, 14 pages.

[17] Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens Van Der Maaten. 2020. Certified data removal from machine learning models. In *Proceedings of the International Conference on Machine Learning*. JMLR.org, Article 359, 11 pages.

[18] Xi He, Ashwin Machanavajjhala, and Bolin Ding. 2014. Blowfish privacy: tuning privacy-utility trade-offs using policies. In *Proceedings of the International Conference on Management of Data (SIGMOD)* (Snowbird, Utah, USA). Association for Computing Machinery, New York, NY, USA, 1447–1458.

[19] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards Practical Differential Privacy for SQL Queries. *PVLDB* 11, 5 (2018), 526–539.

[20] Shiva Prasad Kasiviswanathan, Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2013. Analyzing graphs with node differential privacy. Springer-Verlag, Berlin, Heidelberg, 457–476.

[21] Daniel Kifer and Ashwin Machanavajjhala. 2011. No free lunch in data privacy. In *Proceedings of the International Conference on Management of Data (SIGMOD)* (Athens, Greece). Association for Computing Machinery, New York, NY, USA, 193–204.

[22] Daniel Kifer and Ashwin Machanavajjhala. 2014. Pufferfish: A framework for mathematical privacy definitions. *ACM Transactions on Database Systems (TODS)* 39, 1, Article 3 (Jan. 2014), 36 pages.

[23] Benny Kimelfeld. 2012. A dichotomy in the complexity of deletion propagation with functional dependencies. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (Scottsdale, Arizona, USA). Association for Computing Machinery, New York, NY, USA, 191–202.

[24] Ios Kotsogiannis, Yuchao Tao, Xi He, Maryam Fanaeepour, Ashwin Machanavajjhala, Michael Hay, and Gerome Miklau. 2019. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12, 11 (2019), 1371–1384.

[25] Sebastian Kruse and Felix Naumann. 2018. Efficient discovery of approximate dependencies. *PVLDB* 11, 7 (March 2018), 759–772.

[26] Shulei Kuang, Honghui Yang, Zijing Tan, and Shuai Ma. 2024. Efficient Differential Dependency Discovery. *PVLDB* 17, 7 (2024), 1552–1564.

[27] Gilles M. E. Lafue. 1982. Semantic Integrity Dependencies and Delayed Integrity Checking. In *Proceedings of the International Conference on Very Large Databases (VLDB)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 292–299.

[28] Changchang Liu, Supriyo Chakraborty, and Prateek Mittal. 2016. Dependence Makes You Vulnerable: Differential Privacy Under Dependent Tuples. In *NDSS*, Vol. 16. The Internet Society, San Diego, CA, 21–24.

[29] Ester Livshits, Alireza Heidari, Ihab F. Ilyas, and Benny Kimelfeld. 2020. Approximate denial constraints. *PVLDB* 13, 10 (June 2020), 1682–1695.

[30] Neha Makhija and Wolfgang Gatterbauer. 2025. Is Integer Linear Programming All You Need for Deletion Propagation? A Unified and Practical Approach. *PVLDB* 18, 10 (2025), 2667–2680.

[31] Frank McSherry and Kunal Talwar. 2007. Mechanism Design via Differential Privacy. In *Proceedings of the Symposium on Foundations of Computer Science*. IEEE Computer Society, USA, 94–103.

[32] Shubhankar Mohapatra, Amir Gilad, Xi He, and Benny Kimelfeld. 2025. Computing Inconsistency Measures Under Differential Privacy. *Proceedings of the ACM Management of Data (PACMMOD)* 3, 3, Article 140 (June 2025), 27 pages.

[33] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional dependency discovery: an experimental evaluation of seven algorithms. *PVLDB* 8, 10 (June 2015), 1082–1093.

[34] Primal Pappachan, Shufan Zhang, Xi He, and Sharad Mehrotra. 2022. Don't Be a Tattle-Tale: Preventing Leakages through Data Dependencies on Access Control Protected Data. *PVLDB* 15, 11 (2022), 2437–2449.

[35] Marcel Parciak, Sebastiaan Weytjens, Niel Hens, Frank Neven, Liesbet M. Peeters, and Stijn Vansummeren. 2024. Measuring Approximate Functional Dependencies: A Comparative Study. In *Proceedings of the International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 3505–3518.

[36] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2019. Discovery of Approximate (and Exact) Denial Constraints. *PVLDB* 13, 3 (2019), 266–278.

[37] Eduardo H. M. Pena, Fabio Porto, and Felix Naumann. 2024. Discovering Denial Constraints in Dynamic Datasets. In *Proceedings of the International Conference on Data Engineering (ICDE)*. IEEE, Utrecht, Netherlands, 3546–3558.

[38] Xuebin Ren, Liang Shi, Weiren Yu, Shusen Yang, Cong Zhao, and Zongben Xu. 2022. LDP-IDS: Local Differential Privacy for Infinite Data Streams. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. Association for Computing Machinery, 1064–1077.

[39] Supreeth Shastri, Vinay Banakar, Melissa Wasserman, Arun Kumar, and Vijay Chidambaram. 2020. Understanding and Benchmarking the Impact of GDPR on Database Systems. *Proceedings of the VLDB Endowment (PVLDB)* 13, 7 (mar 2020), 1064–1077.

[40] Chenghong Wang, Johes Bater, Kartik Nayak, and Ashwin Machanavajjhala. 2021. DP-Sync: Hiding Update Patterns in Secure Outsourced Databases with Differential Privacy. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. Association for Computing Machinery, New York, NY, USA, 1892–1905. https://doi.org/10.1145/3448016.3457306

[41] Renjie Xiao, Zijing Tan, Haojin Wang, and Shuai Ma. 2022. Fast Approximate Denial Constraint Discovery. *PVLDB* 16, 2 (2022), 269–281.